AgentISA: Formal Instruction Set Design for LLM Agents

Anonymous Author(s)

Abstract

Large Language Models increasingly run as autonomous agents, yet ad hoc prompt engineering and per-provider function calling leave behavior hard to specify, analyze, and optimize. We present AgentISA, an instruction set architecture making agent behavior a first-class compilable artifact: (1) a formal instruction set with compositional semantics, a type system, and pre/post-conditions, bracketed at runtime by pre-dispatch validation; a shipped bounded verifier for pure instructions is unit-tested but not exercised by the benchmarks; (2) Agentic Calling, lifting provider function calling into reusable, cacheable instructions; (3) instruction learning mining recurring trace patterns into developer-gated composite replays; (4) a compiler and runtime applying classical optimizations—fusion, dead-code elimination, pipeline scheduling, hot-path specialization—to instruction streams.

A reproducible mock-backend microbenchmark drives the real implementation: result caching cuts simulated API calls by 59.5–79.6% on synthetic Zipf workloads (2,000 tasks, 5 seeds); static compilation removes 34.2% of effective instructions (35.4% at length 1,000) on planted optimization targets at disclosed rates (Section 5.1.2); dynamic hot-path optimization shrinks repeated traces by 65.8%. A plain exact-response cache matches this reduction to the digit; its distinct value is reducing the program before any request is issued. Live-gateway conformance checks across four model aliases confirm the implementation matches its backend-free specification—cache statistics equal the computed expectation on the two caching-protocol aliases; the compiled, hot-path-adopted stream issues zero new gateway calls with byte-identical artifacts (live-model-checked in one rerun); they test the implementation, not model variation. A four-task characterization bounds them at 12.5–25.0% per-task compile shrink; a 100-trace census characterizes generator-produced traces (Section 5.10; it does not establish realism); 43 multi-agent tests provide functional validation. These are controlled-setting results, not production end-to-end measurements. Result caching, the benchmark-exercised static passes, trace re-optimization, and six opt-in mechanisms ship; specified-only components carry no claim (Section 5.11).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

AgentISA'26, Virtual Event

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

CCS Concepts

• **Software and its engineering** → >**Compilers**; • **Theory of computation** → >**Program reasoning**; • **Computing methodologies** → >**Artificial intelligence**; • **Computer systems organization** → >**Architectures**.

Keywords

Instruction Set Architecture; Large Language Models; Compiler Optimization; Multi-agent Systems; Formal Methods; Result Caching

ACM Reference Format:

Anonymous Author(s). 2026. AgentISA: Formal Instruction Set Design for LLM Agents. In *Proceedings of AgentISA 2026 (AgentISA'26).* ACM, New York, NY, USA, 20 pages.

1 Introduction

Recent advances in Large Language Models (LLMs) have produced systems with strong understanding, generation, and—through instruction tuning and human feedback—instruction-following capabilities [1, 2, 3, 4, 5, 6], and a rapidly growing body of work couples them with tools, memory, and control loops to build autonomous agents [9, 10, 11, 12]. Yet the engineering substrate beneath these agents remains ad hoc: behaviors are specified as prompts and provider-specific function-calling schemas—difficult to specify precisely, analyze formally, or optimize systematically.

1.1 Background and Motivation

Function calling—selecting an API from a declared tool set and emitting structured arguments—has become the dominant interface between LLMs and external systems [13, 14, 15], consolidated by standardization efforts such as the Model Context Protocol [18] and measured by harnesses such as the Berkeley Function Calling Leaderboard [17]. However, the unit of interaction remains the individual API call: each call is dispatched, priced, and forgotten independently, even when an agent repeatedly executes structurally identical work.

This per-call statelessness contrasts with how computing systems historically handle repetitive execution: classical compilers and hardware translate recurring patterns into cheaper forms—instruction fusion, dead-code elimination, caching, trace-based dynamic optimization [33, 34, 45]. Our motivating hypothesis is that these techniques apply to LLM agent workloads: if agent behavior is expressed as an explicit instruction stream rather than opaque prompts, recurring patterns become visible to a compiler and optimizable with well-understood, semantics-preserving transformations.

1.2 Challenges in Current LLM-based Agent Systems

Five structural challenges motivate AgentISA:

1. **Per-call statelessness of tool interfaces.** Mainstream function-calling protocols treat each call as an isolated event [13, 14, 18]: the provider has no notion of a *program* being executed, repeated identical work incurs repeated cost, and caching, where present, is an afterthought rather than a property of the execution model.

2. **Experience that does not accumulate as compiler-visible instruction structure.** Paradigms such as Reflexion [10] and Voyager [12] accumulate experience as verbal reflections or inference-time skill libraries—not reified as reusable, statically optimizable instructions a compiler can fuse, schedule, or eliminate.

3. **Absence of a shared behavioral contract.** Each multi-agent framework defines its own abstractions for roles, messages, and workflows [19, 20, 21, 22, 23, 24]; no instruction-level contract spans frameworks, fragmenting tooling and cross-framework analysis.

4. **Cost pressure at the wrong layer.** FrugalGPT [28] shows LLM inference cost is a first-order concern, and serving systems attack it below the API boundary [38, 39, 40, 41]; what is missing is the complementary, above-the-API layer that reduces the *number and size* of calls an agent issues in the first place.

5. **Integration complexity across providers.** Portability is handled at the library level [26, 27]: logic written against one provider's function-calling surface does not transfer cleanly to another, and capability differences are hard to reason about systematically [17].

1.3 Research Objectives and Contributions

This paper introduces AgentISA, which addresses these challenges by making agent behavior a first-class compilable artifact. Our contributions are:

1. **A formal instruction set** for LLM agents with compositional semantics, a type system, and pre/post-conditions on every instruction (Section 3.1). Two deliberately modest sanity results serve as design rationale: Turing completeness (Theorem 3.1—a sanity check that the instruction set does not constrain expressiveness; it equally entails undecidability of static properties in general, which is why the Section 3.3.2 checker is bounded) and type safety for the pure core (Theorem 3.2), fixing the well-formedness obligation the offline bounded checker discharges for pure instructions (Section 3.3.2) and the runtime brackets with pre-dispatch validation.

2. **Agentic Calling**, a mechanism lifting provider-specific function calling into reusable, cacheable, semantically specified agent instructions, with a model-adapter layer for cross-provider portability (Section 3.2).

3. **Trace-based instruction learning**, which mines recurring instruction patterns from interaction traces and promotes validated recurring patterns into candidate instructions whose `execute` bodies replay the mined constituent subsequence by sequential composition over a shared context (Section 3.3; replay reproduces the recorded subsequence's outputs on the recorded inputs and, via shipped schema-checked binding, on unseen parameter values—constructor-baked parameters excepted, as the section states; the generalization is quantified: 1,050/1,050 unseen-parameter replays match the oracle exactly, Section 5.11).

4. **A compiler and runtime** implementing fusion, dead-code elimination, parallel-grouped scheduling, and dynamic hot-path specialization for agent instruction streams (Sections 3.4 and 4), together with an opt-in tree-pattern instruction selector (shipped, Section 4.1.1; benchmarked in Section 5.11: covering cost at or below the default heuristic's on 20/20 random IR trees, mean ratio 0.361, the gap synthetic-pricing-dependent) and cycle-accurate pipeline scheduling (specified, Section 4.1.2), with semantic-preservation arguments for the implemented rewrites (Theorems 3.4–3.6, proofs in Appendix C).

5. **An empirical evaluation** of the implemented system, with full methodology disclosure. On synthetic Zipf workloads (5 random seeds each), caching reduces simulated API calls by 59.5–79.6% depending on skew, static compilation reduces the effective instruction count by 34.2% ± 1.8% (length 200), and dynamic optimization shrinks repeatedly executed traces by 65.8% ± 4.6%; real-gateway experiments (Sections 5.7–5.7.1) validate the cache and compiled pipeline on a live gateway, a four-task characterization (Section 5.8) composes the mechanisms on realistic streams, a 100-trace vocabulary census (Section 5.10) documents the cross-task reuse structure of generator-produced traces, and six mechanism experiments (Section 5.11) exercise the shipped opt-in components.

6. **Multi-agent coordination infrastructure** with capability-threshold agent selection (exercised with injected scores; the static metric extractor ships unwired, Section 3.5), execution planning, fallback strategies, and a security monitor enforcing authentication, authorization, resource-access, and data-sensitivity policies (Sections 3.5 and 5.6), validated by 43 passing functional tests; an opt-in fail-closed security mode now also ships, and its paired posture matrix (1,000 evaluation tasks per mode) shows it denying the 400 policy-gap tasks the fail-open default admits with zero false denials on well-specified tasks (Section 5.11).

Our empirical claims are scoped accordingly: Sections 5.2–5.5 exercise the real `agentflow.core.isa` implementation against a mock LLM backend on synthetic workloads; Sections 5.7–5.8 extend the validation at reduced scale on realistic streams; Section 5.10's census covers generator-produced traces, Section 5.11's mechanism experiments run on declaration-injection pools and synthetic trees, and Section 5.12 replays 1,960 recorded model trajectories—still no production end-to-end measurements (threats to validity in Section 5.9).

2 Related Work

2.1 Foundation LLMs and Function Calling

Modern LLMs are autoregressive transformers trained at large scale [1, 2, 3, 4], with instruction tuning and reinforcement learning from human feedback making them reliable instruction followers [5, 6]. Function calling constrains their output distribution to a structured output space, $F(x) = \text{argmax}_{f \in \mathcal{F}} P(f|x, \mathcal{F})$, where $\mathcal{F}$ is the set of declared functions and $F(x)$ the selected call. Frontier and open models alike expose this capability with provider-specific schemas [2, 3, 4]; AgentISA does not modify this layer but treats any such interface as a *backend* targeted by a common instruction set, as a compiler targets multiple processor backends [33].

The cost structure of this layer motivates our work. If each call carries latency τ_i plus a context-switching overhead ω_i (prompt reconstruction, serialization, cache misses; the symbol δ is reused in Appendix B for per-instruction transition functions—the two are unrelated), then n similar calls cost $C(n) = \sum_{i=1}^{n}(\tau_i + \omega_i)$.

Definition 2.1 (Function Call Overhead): The overhead of executing n function calls is $C(n)$, the sum of per-call latencies and per-call context reconstruction costs. When repeated calls carry identical semantic inputs and satisfy the cache-validity conditions of Section 3.2, the repeated context-construction cost ω_i is avoidable—which is precisely the cost AgentISA's caching and fusion are designed to remove. Structural similarity alone does not make a re-invocation waste: if an input's dependencies have changed or the call probes external state, re-execution has semantic meaning; the cache-validity conditions, not structural identity, license reuse. Under the result cache, the measured quantity is $C_{cached}(n) = \sum_{i \in miss}(\tau_i + \omega_i)$ and Section 5.2 reports $|miss|/n$ as the call-reduction rate.

2.2 Reasoning and Agent Paradigms

A second line enhances reasoning at prompt time: chain-of-thought [7] and zero-shot reasoning [8]; ReAct's interleaved reasoning and action [9]; Reflexion's verbal self-critique [10]; Tree of Thoughts' deliberative search [11]; Voyager's open-ended skill acquisition [12]. These paradigms define *what* an agent should do at the cognitive level; AgentISA defines *how* the resulting behaviors are represented, compiled, and executed as instructions—a ReAct loop becomes an instruction sequence whose tool invocations are fused, cacheable agentic calls.

2.3 Tool Calling and Interface Standardization

Toolformer trains models to interleave tool calls with text [13]; Gorilla connects models to large API corpora [14]; ToolLLM scales tool coverage to 16,000+ real-world APIs [15]; HuggingGPT orchestrates model hubs [16]. On the standardization side, the Model Context Protocol specifies how tools and context are exposed [18], and the Berkeley Function Calling Leaderboard evaluates function-calling accuracy cross-model [17]. These efforts standardize the *interface*; AgentISA standardizes the *execution model above it*: an AgentISA program compiled for two providers differs only in the final translation step (Section 3.2.3), and interface-level standards alone cannot fuse or eliminate calls.

2.4 Multi-Agent Frameworks

AutoGen frames agent applications as multi-agent conversations [19]; MetaGPT assigns standardized roles and workflows [20]; AgentVerse facilitates multi-agent collaboration and studies emergent behaviors [21]; ChatDev organizes agents as a software-development organization [22]; CAMEL studies communicative agent societies [23]; AutoGPT popularized autonomous goal-driven agents [24]. AgentISA contributes a different artifact to this space: not a framework or orchestration philosophy but an instruction-level substrate—capability profiles, coordination instructions with barrier semantics, and a security monitor providing capability-based agent selection, per-stage fallback planning, and policy enforcement—that any such framework could target (Section 3.5, Section 5.6).

2.5 Prompt Compilation and Cost Optimization

Closest to our compilation perspective is DSPy, which compiles declarative prompt programs into optimized pipelines via prompt search and fine-tuning [25]; LangChain [26] and LlamaIndex [27] provide library-level chaining and indexing abstractions; FrugalGPT reduces cost through model cascades [28]. AgentISA differs in the level at which optimization operates—DSPy optimizes *prompts*, FrugalGPT *model selection*, AgentISA *instruction streams*: fusion, dead-code elimination, parallel grouping (the shipped pipeline scheduling), result caching, and hot-path specialization—implemented and measured (Sections 3.4, 4, and 5)—plus opt-in tree-pattern selection (Section 4.1.1) and specified cycle-accurate scheduling. These transformations [33] compose with, rather than replace, prompt- and model-level optimization.

2.6 Benchmarks for Agent Systems

AgentBench evaluates LLMs as agents across diverse environments [29]; GAIA tests general assistant abilities [30]; PlanBench probes planning and reasoning about change [31]; SWE-bench measures resolution of real-world GitHub issues [32]. These benchmarks evaluate *end-to-end capability*; AgentISA's evaluation measures *subsystem behavior under controlled conditions*—call reduction, instruction-count reduction, trace shrinkage (Section 5). The two are complementary: instruction-level metrics can attribute end-to-end gains to specific mechanisms when such gains are measured, and are reproducible without live model access.

2.7 Classical Compiler and Architecture Techniques

AgentISA's optimization machinery is deliberately classical: tree-pattern instruction selection with dynamic programming, peephole optimization, and data-flow-based dead-code elimination [33]; pipelining, hazard detection, and resource-constrained scheduling [34]; the open, well-specified instruction-set discipline of RISC-V [35]; type safety via progress-and-preservation [36];

formally verified compiler correctness in the CompCert tradition [37]. Our contribution is not a new technique but the demonstration that this machinery transfers to LLM agent workloads, with the semantic-preservation arguments adapted to instructions with pre/post-conditions (Appendix C).

2.8 LLM Serving Systems

Below the API boundary, the serving community has made inference dramatically cheaper: PagedAttention [38]; Orca's continuous batching [39]; FlashAttention's IO-aware attention [40]; Mooncake's KVCache-centric disaggregation [41]; inference-cost energy analyses [42]. This line and AgentISA are orthogonal layers of the same stack: serving systems make each call cheaper; AgentISA reduces how many calls an agent makes and how much redundant work each carries—the two compose (whether they multiply is unmeasured).

2.9 Research Gaps and Our Contributions

In summary, the literature leaves four gaps that AgentISA addresses:

1. **No instruction-level contract for agent behavior.** Interfaces are standardized [13, 14, 18] and frameworks numerous [19, 20, 21, 22, 23, 24], but no shared instruction set spans providers and frameworks.

2. **No compiler for agent behavior.** Prompt-level compilation exists [25] and model-level cost reduction exists [28], and *caching itself is well-trodden ground*—at the response level (GPTCache [46]), the KV-prefix level (RadixAttention/SGLang [47]), and the provider prompt-caching level. None of these layers expose an *instruction-level* contract: caching there optimizes model serving—invisible to the agent program's structure, without effect on it. Instruction-level fusion, elimination, and scheduling—with cacheability a declared property of the instruction set—have not, to our knowledge, been demonstrated.

3. **No semantics-preserving account of agent optimization.** Serving work verifies kernels and schedulers, not agent-level transformations; we provide progress-and-preservation-style arguments for ours (Theorems 3.2, 3.4, 3.5, 3.6).

4. **No reproducible instruction-level evaluation.** Existing benchmarks measure end-to-end capability [29, 30, 31, 32]; we contribute a seed-controlled evaluation: a microbenchmark suite with fully disclosed synthetic workloads and mock-backend instrumentation, extended by real-gateway and system-level validations (Section 5).

3 AgentISA Design

Building on Sections 1 and 2, we present the AgentISA design: the core architecture (Section 3.1), the Agentic Calling framework (Section 3.2), trace-based instruction learning (Section 3.3), instruction organization and optimization (Section 3.4), and the capability-assessment layer (Section 3.5).

3.1 Core Architecture

3.1.1 Instruction Set Specification. The AgentISA instruction set is formally defined as a tuple:

Definition 3.1 (AgentISA Core): AgentISA $= (I, T, \Sigma, \Delta, \Phi)$, where I is the set of instructions, T the type system, Σ the state space, Δ the transition function set, and Φ the verification function set.

Following the conventions of classical instruction set design [33, 34, 35], we define the basic instruction format:

```
INSTRUCTION = {
    opcode: OPCODE,
    args: ARGS[],
    precondition: PRED,
    postcondition: PRED,
    context: CONTEXT
}
```

The instruction set is categorized into four fundamental classes: **control flow** ($\mathcal{C} = \{seq, branch, loop, call\}$), **state management** ($\mathcal{S} = \{load, store, update, cache\}$), **LLM interaction** ($\mathcal{L} = \{query, response, context\}$), and **resource management** ($\mathcal{R} = \{allocate, free, monitor\}$).

Two sanity results anchor the definition; neither introduces new machinery [33, 36]—Theorem 3.1 certifies that the instruction set does not constrain what agent programs can express, and Theorem 3.2 fixes the well-formedness obligation discharged by the offline bounded checker for pure instructions (Section 3.3.2) and bracketed at runtime by pre-dispatch context validation (Section 3.2.3).

Theorem 3.1 (Turing Completeness): The AgentISA instruction set is Turing complete.

Proof sketch: a translation from WHILE programs—known Turing equivalent [44]—to instruction sequences, by simulation: state management instructions encode an unbounded store, control-flow instructions encode composition, branching, and iteration. The full construction is in Appendix A.

3.1.2 Type System and Semantics. The type system T extends the simply-typed discipline of Pierce [36] with dependent-type constructions (Π, Σ, μ):

Definition 3.2 (AgentISA Type System):

$$T ::= \text{Base} \mid \Pi(x : T_1).T_2 \mid \Sigma(x : T_1).T_2 \mid \mu X.T$$

where Base types include {Bool, Int, String, Float}, Π/Σ represent dependent function/pair types, and μ recursive types. The semantic rules follow standard operational semantics [43] (a context rule and β-reduction).

The pure core carries a standard type-safety argument—progress and preservation, in the style of Pierce [36]—discharging the well-formedness role fixed in Section 3.1 and extended to the shipped instruction set by the bounded checker of Section 3.3.2:

Theorem 3.2 (Type Safety for the Pure Core): Well-typed programs in the pure core calculus of Appendix B—the λ-fragment (base types, dependent Π/Σ types, recursion) plus the composition operators applied to instructions carrying typed pre/post-conditions—cannot "go wrong", under the per-instruction validity obligation of Definition 3.5 (stated in Section 3.3.2 below;

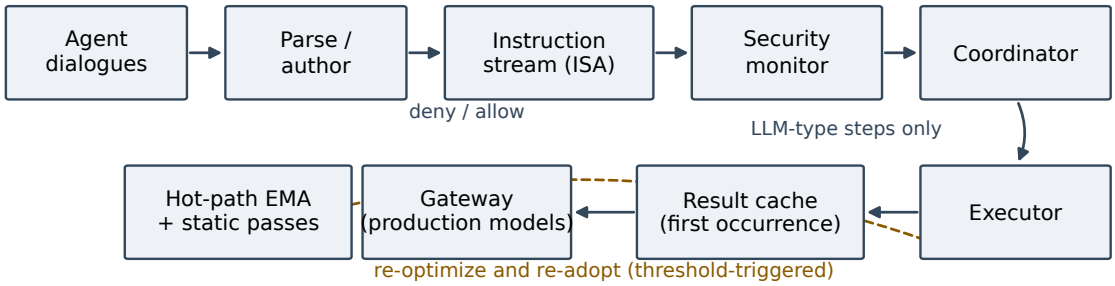


Figure 1: AgentISA runtime architecture. Authored streams pass a security gate and a coordinator that routes LLM-type steps through a result cache (first occurrence only) against the production gateway; a hot-path detector and the static optimizer re-optimize and re-adopt the stream (threshold-triggered). Sections 5.7-5.7.1 measure each property on the real code path.

pre/post-conditions), with transition functions assumed total on precondition-satisfying states (Appendix B.7). Effectful instruction classes are embedded via their typed pre/post-conditions and covered to the extent their preconditions are checkable.

Proof sketch: By the standard two-lemma technique—progress ($\forall e, T.\ \vdash e : T \implies \text{value}(e) \vee \exists e'.\ e \to e'$) and preservation ($\forall e, e', T.\ \vdash e : T \wedge e \to e' \implies \vdash e' : T$)—as presented in Pierce [36], both proved by induction over Appendix B's merged judgment system, with a joint argument for the composition cases: progress from the Definition 3.5 obligation plus totality, preservation from postcondition typing, composition via the chained invariant $post(i_1) \Rightarrow pre(i_2)$ (Appendix B).

Scope of the totality premise. Totality rules out exactly the failure modes that dominate live LLM calls—timeouts, refusals, malformed replies, and `max_tokens` truncation (the Section 5.7.1 census records ceiling-binding on 22/22 calls at the 512 budget on the reasoning alias). Theorem 3.2 therefore constrains the composition and dataflow layer only; an error-value semantics (δ returning a typed Err, progress weakened to "value, Err, or reducible") that would make totality constructive remains future work (the premise is recorded in Appendix B.7).

3.1.3 LLM Integration Interface. The LLM integration interface is a protocol playing the same role for agent backends that the Model Context Protocol plays for tool exposure [18]: $\mathcal{P}_{LLM} = (Q, R, C, \psi)$, where Q is the query space (JSON Schema), R the response space (JSON Schema), C the context-management surface (today the context-window size), and ψ the state mapping Map<AgentState, LLMContext>—corresponding to the interface conventions of modern function-calling providers [3, 18].

3.2 Agentic Calling Framework

3.2.1 Function Calling to Agentic Calling Transformation. The transformation from function calling to Agentic Calling is formalized as **Definition 3.3** (Agentic Transformation): $\mathcal{T} : F \to A$, where F is the function call space and A the agent instruction space.

$\mathcal{T}$ preserves call semantics **on the dispatch path** by construction: the synthesized instruction records the function signature verbatim and delegates execution to the original call, so $\text{sem}(\mathcal{T}(f)) \equiv \text{sem}(f)$ whenever the cache is bypassed. **On the cached path** preservation holds only under Section 2.1's cache-validity conditions (determinism, no external-state probing, unchanged dependencies); outside them the cache is an intentional semantic change—the mechanism's benefit. Context embedding is content-preserving by the copy construction (Theorem 3.3); that theorem covers the context field only, not call semantics.

The three steps—function signature analysis, context embedding, instruction synthesis—mirror the signature-to-call conventions of function-calling interfaces [13, 14].

3.2.2 Context Preservation and State Management. State management in AgentISA follows a formal state machine model, $M = (S, s_0, \delta, F)$: states S with initial state s_0, transition function δ, and final states F.

Extraction-time context preservation is guaranteed by construction (Theorem 3.3); at runtime it is only monitored:

Theorem 3.3 (Context Preservation): Let ϕ be the context interpretation function (defined compositionally over context *contents*—the keys and values the context carries), and t the extraction transformation that embeds a context c verbatim into an instruction's `context` field. Then, for every $c \in C$: $\phi(t(c)) = \phi(c)$.

Proof: By the definition of t, the extracted instruction carries a verbatim copy of c; since ϕ is defined over context contents and the copy is content-identical, $\phi(t(c)) = \phi(c)$ holds by construction—a *design invariant of the extraction pipeline* (the implementation copies rather than re-encodes). At runtime the shipped checks are constraint-based and bracket the transition—`AgenticTransformer.validate_context` checks the registered context constraints pre-dispatch, `verify_context_preservation(before, after)` re-checks them pairwise across the executed transition (mechanics in Section 3.2.3)—and perform no structural validation of the carried context. Extraction-time preservation is discharged by the copy construction; preservation across an executed transition is *monitored, not guaranteed*, by the runtime checks. ■

3.2.3 Cross-model Compatibility. Cross-model *protocol* compatibility—not behavioral equivalence of models—is achieved through an abstract interface layer (provider-independent instruction representation with provider-specific adapters):

```
Interface ModelAdapter {
    translate: AgentInstruction → ModelSpecificCall      // shipped:
    ↪ translate_instruction
    normalize: ModelSpecificResponse → AgentResponse    // shipped:
    ↪ normalize_response
    validateContext: Context → Boolean                   // shipped:
    ↪ AgenticTransformer.validate_context
}
```

All three operations ship: `translate` is `ModelAdapter.translate_instruction` (with registered type transformations); `normalize` is `ModelAdapter.normalize_response`, folding the two dominant wire shapes—Anthropic (typed `content` blocks, `usage.input_tokens`/`output_tokens`) and OpenAI (`choices[0].message`, `usage.prompt_tokens`/`completion_tokens`)—into a canonical `AgentResponse` envelope (content, tool calls, usage, model, raw), so downstream code never branches on provider syntax; and `validateContext` is `AgenticTransformer.validate_context`, the pre-execution complement of the shipped post-hoc `verify_context_preservation(before, after)`—it evaluates the registered context constraints against a single state before dispatch, applying two-argument predicates reflexively (v, v) (single-argument ones directly) and leaving constraints over absent keys vacuously satisfied, so key *deletion* goes unreported—a blind spot analogous to non-reflexive constraints (e.g. strict growth), which are not detectable pre-execution.

Motivated by cross-model function-calling studies [17] and open protocol efforts [18]—providers differ in surface syntax but can agree on *protocol shape*—the requirement is a **protocol-level contract on the adapter layer**: each adapter m's request and response normalizers, $\text{norm}_{in}(m)$ and $\text{norm}_{out}(m)$, are total on the supported request and response classes, mapping every supported raw request and response into the unified schemas, with declared types preserved and errors classified into a common taxonomy. This is deliberately weaker than semantic-output equivalence—$m_1 = m_2$ would already demand determinism, which no LLM provides—and cross-model *behavioral* comparison is an empirical question at benchmark level (Section 3.5, [17, 29]).

3.3 Trace-Based Instruction Learning

3.3.1 Dialogue Pattern Recognition. Pattern recognition in AgentISA is frequency-based and deterministic (action-signature grouping) rather than grammatical; it is implemented in `agentflow.core.isa.pattern_mining` and `agentflow.core.isa.instruction_learning`. To prevent over-reading: the subsystem is a **discovery-and-replay pipeline, not an autonomous learning loop**. Its five stages are (i) trace-derived pattern mining (this section), (ii) dialogue-derived template extraction (§3.3.2), (iii) candidate validation (support and success-rate thresholds), (iv) *gate-controlled* promotion (`agentflow.core.isa.auto_promotion` edits the optimizer's rule set only through an approval gate; opt-in `auto_approve` removes the human step), and (v) parameterized composite replay. Stages (i)–(v) ship; the stage-(iv) gate defaults to approval.

Definition 3.4 (Instruction Sequence Pattern): A pattern p over the instruction alphabet I is a non-empty contiguous sequence $p = (i_1, \ldots, i_k) \in I^k$ together with two frequency statistics:

- **Support**: $sup(p) = P(p \text{ occurs as a contiguous subsequence in a randomly d}$
- **Confidence**: $conf(p \Rightarrow j) = P(i_{t+1} = j \mid (i_{t-k+1}, \ldots, i_t) = p)$

A pattern is *frequent* if $sup(p) \geq \sigma_{min}$ and *predictive* if $\max_j conf(p \Rightarrow j) \geq \gamma_{min}$, for thresholds $\sigma_{min}, \gamma_{min} \in (0, 1]$.

Pattern recognition proceeds in three stages:

1. **Frequent-sequence mining.** Candidate subsequences are enumerated from execution traces with occurrence counts; successor statistics are accumulated alongside, and sequences below σ_{min} are pruned together with their statistics—so confidence is estimated only over frequent sequences. The output is the set of frequent, predictive patterns.

2. **Behavior grouping.** The shipped `PatternRecognizer` groups dialogues deterministically by *action signature*—the sorted set of system-turn contents—and promotes each sufficiently supported bucket into a pattern. (An earlier prototype and why it was abandoned: artifact notes, supplementary F.8.) Promotions split by source: trace-mined promotions execute by **composite replay**—the `execute` body re-runs the recorded constituent sequence, with its dataflow edges, in mined order against a shared context, merging outputs and returning the keys its `produces_X` postconditions declare—while dialogue-mined templates run an attached `executor` if one exists (dict results checked against the declared `produces_X` postconditions) and otherwise raise `NotImplementedError` rather than silently no-opping. Promotion is deterministic (pattern identity derives from the mined instruction-id sequence), and replay is plain sequential composition, so a learned instruction reproduces its recorded subsequence's outputs on the recorded inputs; on unseen parameter values the binding above re-runs the recorded operator *instances*, so parameters baked into an operator's constructed state cannot be rebound, and the schema is observational—mixed or unseen types fall back to `object`. The mining-to-optimization path now exists: an approved proposal registers the fused instruction and installs a name-level peephole rule into the optimizer's rule set, gated by default on a developer's approval.

3. **Pattern validation.** Each candidate is checked for significance—support above σ_{min} and, for dialogue-derived patterns, an acceptable success rate—before promotion to a candidate instruction.

The design is deliberately statistical—agent traces are short and noisy, and frequency statistics degrade gracefully where grammar induction would overfit—and it composes with the compiler: a frequent, verified pattern is exactly the promotion candidate of Section 3.4.4.

3.3.2 Instruction Optimization and Verification. Optimization is coupled to verification: where verified-compilation efforts such

as [37] verify the transformations of a whole compiler, AgentISA's verification targets a different granularity—the validity of individual instructions and the obligations of individual rewrites:

Definition 3.5 (Instruction Verification): An instruction i is valid if $\forall s \in S : \text{pre}(i, s) \implies \text{post}(i, \text{exec}(i, s))$.

The shipped checker (`agentflow.core.isa.bounded_checker`) enumerates the candidate input states derived from the declared parameter constraints—exhaustively for finite domains such as `bool` and enum parameters, and as a bounded, deterministically sampled set (cap 256) otherwise—making it *bounded*, not complete. For every pure instruction it filters states by well-typedness and preconditions, executes it, and evaluates three obligations per surviving state—progress, preservation (declared outputs present and well-typed), and the postcondition itself. Side-effectful instructions are never executed and are reported out of scope; their obligation is delegated to authoring-time tests and review. Theorem 3.2 is conditional on this obligation, Theorems 3.4–3.6 carry the separate rewrite obligations of Appendix C.7, and beyond the enumeration bound the condition remains substantive rather than discharged (Appendices B.7 and C.7)—formally, the checker establishes $\forall s \in S_{bounded} \subseteq S : \text{pre}(i, s) \implies \text{post}(i, \text{exec}(i, s))$, where $S_{bounded}$ is the explored subset; completeness is traded for tractability—the standard bounded-exploration compromise, with the operational-semantics treatment of instruction meaning following [36].

The guarantee therefore comes in two explicitly separate layers. **Layer 1 (theory, conditional):** Theorem 3.2 and the type-safety results of Appendix B hold for every program whose instructions satisfy Definition 3.5 *on all of S*, under the total-definedness premise of Appendix B.7—a mathematical statement about a proof obligation, not about the checker. **Layer 2 (implemented verification):** the shipped checker discharges that obligation only on $S_{bounded}$ and only for pure instructions, as described above. Success of the bounded check is evidence toward, not a proof of, Layer 1's premise; no claim in this paper equates the two.

Theorem 3.4 (Optimization Soundness): Every atomic rewrite of the pass pipeline—fusion (Appendix C.2's rule, applied by the combining pass), dead-code elimination, and instruction combining (Section 3.4.4)—preserves the denotation of the instruction sequence it transforms: $\text{sem}(r(i)) \equiv \text{sem}(i)$, read as the partial equivalence of C.1.

Proof sketch: By structural induction on instruction composition. The base case shows each atomic rewrite preserves the denotation of a single instruction; the inductive step lifts preservation through each composition operator of Definition 3.6. The full proof, with the per-rewrite case analysis, is in Appendix C.

The shipped checker (`agentflow.core.isa.bounded_checker.verify_instruction`) implements Definition 3.5 on the bounded state space and reports violations structurally (listing in Appendix D.11 of the supplementary material).

3.4 Instruction Organization and Optimization

3.4.1 Instruction Composition. AgentISA supports hierarchical instruction composition through a formal composition algebra:

Definition 3.6 (Instruction Composition): Given instructions $i_1, i_2 \in I$, we define four composition operators—Sequential $i_1 \circ i_2$, Parallel $i_1 \parallel i_2$, Conditional $i_1 \triangleright i_2$, and Iterative i_1^*—whose denotations (sequential execution; parallel interleaving under disjoint state footprints; guarded execution of i_2 on i_1's triggering outcome; zero-or-more iterations of i_1) are fixed in Appendix C.1.

Composition follows algebraic properties. Properties 1 and 3 hold for all instructions; Property 2 is a *conditional* law—purity and pairwise footprint disjointness together do *not* suffice, and idempotence of i_1 must be assumed separately:

1. **Associativity** (all instructions, up to denotation): $(i_1 \circ i_2) \circ i_3 = i_1 \circ (i_2 \circ i_3)$.
2. **Distributivity** (conditional on idempotent i_1 and pairwise-disjoint footprints): the law $i_1 \parallel (i_2 \circ i_3) = (i_1 \parallel i_2) \circ (i_1 \parallel i_3)$ holds under two premises—**(a)** idempotence of i_1 on precondition-satisfying states and **(b)** pairwise footprint disjointness—neither of which follows from purity (necessity arguments and counterexamples: supplementary C.3). The shipped compiler does not rely on the law; it is stated for the algebra's completeness.
3. **Identity**: $i \circ \epsilon = \epsilon \circ i = i$.

   where ϵ is the empty (identity) instruction.

Remark (implementation consequence): distributivity also fails for side-effectful instructions, for the same double-execution reason. Accordingly, parallel grouping is gated on purity plus disjoint dependencies and disjoint outputs, and `side_effects=True` instructions are excluded from all such rewrites.

3.4.2 Instruction Pipeline Design. We formalize the instruction pipeline as a directed acyclic graph (DAG):

Definition 3.7 (Pipeline DAG): $P = (V, E, \lambda, \delta)$, where V are the pipeline stages, E the dependencies between stages, λ the stage latency function, and δ the data transfer function.

The pipeline's stages (FETCH → DECODE → SCHEDULE → EXECUTE → COMMIT, with hazard detection, dependency tracking, and resource allocation maps; listings in Appendices D.2–D.3 of the supplementary material) are organized around this DAG.

Theorem 3.5 (Pipeline Correctness): For any valid instruction sequence s and any schedule S satisfying conditions S1-S4 of Appendix C.5—each instruction dispatched exactly once, dependencies respected, dispatch groups pairwise non-interfering, commit in program order—pipeline execution preserves program semantics.

The shipped dispatch implements the unit-slot case of S1–S4; the multi-period schedule remains conditional on the stated premise.

Proof: By induction over the dispatch groups of S in commit order, under the invariant that the architectural state after committing each prefix of groups equals the sequential state after the same instructions; S1–S4 supply exactly-once dispatch, dependency order,

group commutativity, and in-order commit (details in Appendix C.5).

3.4.3 Instruction Selection and Scheduling. Instruction selection follows a tree-pattern matching approach:

Definition 3.8 (Selection Pattern): $\pi = (\tau, C, \text{cost}, \text{code})$, where τ is a tree pattern over the IR (unrelated to the type-system symbol T of Definition 3.2), C the context constraints on the surrounding state, and cost/code the cost function and generated code.

Given an IR tree t to cover, the selection process is formalized as $\text{Select}(t) = \arg\min_{\pi \in \Pi}\{\text{cost}(\pi) \mid \text{matches}(\pi.\tau, t) \wedge \text{satisfies}(t, \pi.C)\}$.

Definition 3.8's optimal selection ships as the opt-in DP selector of Section 4.1.1; the default remains the heuristic matcher run by Section 5's benchmarks.

Scheduling is formalized as the classical delay-minimization problem under dependency and capacity constraints: minimize the total stall subject to dependency order ($\text{start}(i_2) \geq \text{end}(i_1)$ for every dependency pair) and to capacity at every time (formulation in Appendix E.1 of the supplementary material)—the specified, not shipped, scheduler's objective.

3.4.4 Static and Dynamic Optimization. **Static optimization.** Static optimization employs several techniques:

1. **Peephole Optimization**: $\text{PH}(i_1, \ldots, i_n)$ returns $\text{opt}(i_1, \ldots, i_n)$ if a matching pattern exists, otherwise the window is left unchanged.
2. **Dead Code Elimination**: $\text{Live}(i) = \text{Use}(i) \cup (\text{Live}_{\text{out}}(i) - \text{Def}(i))$. The two shipped passes approximate this fixpoint at different conservatism levels: the *compiler* pass (formal streams) removes a side-effect-free instruction when no live consumer depends on its id or any declared output, while the *trace optimizer* pass (executable instructions) keeps any instruction declaring at least one output—executable traces lack the formal dependency annotations that make output-consumption analysis exact. Section 5.3 exercises the former; Section 5.4 the latter.
3. **Instruction Combining** (the named candidate rules—textbook mnemonics, illustrative only, not shipped opcode names; listed in Appendix D.10 of the supplementary material—are specified and not registered: Section 4.1.3's only shipped write path into the rule set is the promotion gate of Section 3.3.1; the built-in pairwise compatibility combining is active, and it is what the hot-path trace re-optimization of Section 5.4 applies).

Dynamic optimization. Dynamic optimization in the shipped runtime is **repetition-triggered and threshold-based**; it does not (yet) respond to load or performance signals:

1. **Hot Path Detection** (implemented, two shipped modes): the default `exact` mode keeps a monotone counter per exact instruction sequence (`path_id` = the ordered join of instruction ids) and flags a path hot when its count exceeds a configurable threshold *and* its most recent recorded success rate is at least a stability bound (0.9 by default), $\text{Hot}(p) \iff \text{count}(p) > \theta \wedge \text{success}(p) \geq \alpha$—no windowing, no load signal.

   The opt-in `ema` mode instead triggers on recent execution share, over bounded per-path state (Section 4.2.2).
2. **Trace Optimization** (implemented): once hot, the cached sequence is re-optimized by the static pass machinery—peephole, dead-code elimination, instruction combining—bounded to a fixed number of passes; the runtime then serves the shrunken form for subsequent executions (demonstrated end-to-end in Section 5.8).
3. **Trace Formation and Adaptive Recompilation** (specified, not implemented): sub-trace extraction from observed frequency profiles, and performance-triggered recompilation (`should_recompile`: execution count above threshold *and* observed performance below expectation) remain design-level. The shipped runtime re-optimizes the whole cached sequence once per recording after the threshold is crossed, with no performance feedback loop.

Two consequences of the shipped design now have shipped remedies the default does not enable: exact keying means trace diversity (similar-but-not-identical sequences) never accumulates a shared count—the `ema` trigger relaxes the threshold to recent share but stays per-path keyed (residuals in Sections 4.2.2 and 6.1)—and hot-path state, formerly unbounded, is bounded by an LRU cap on tracked paths (`max_tracked_paths`, 1,000 by default) that evicts least-recently-recorded paths, hot ones included.

3.4.5 Verification and Analysis. Optimization correctness is argued through:

Theorem 3.6 (Pass-Pipeline Soundness, extending Theorem 3.4): For any optimization transform T composed of the pass pipeline's rewrites, $\forall p \in \text{Program} : \text{sem}(T(p)) \equiv \text{sem}(p)$.

Proof sketch: semantic preservation for each atomic transformation (the atomic-rewrite lemma, Appendix C.2) composes across transformations by the compositionality lemma (Appendix C.3); hazard-free pipeline execution is *not* part of this theorem—it is Theorem 3.5 (Appendix C.5), invoked separately; this theorem's contribution is the rewrite layer.

Performance Model (specified; the benchmarks of Section 5 measure latency and token counts directly and do not evaluate it): the modeled cost of a program sums, per instruction, base cost plus overhead minus an optimization gain that aggregates each optimization's benefit weighted by its application rate (both formulas in Appendix E.1 of the supplementary material).

3.5 Capability Assessment

Agents are heterogeneous: an agent may be strong at reasoning but weak at tool use, or vice versa. To make such differences explicit—and to give the multi-agent coordinator of Section 5.6 a principled basis for agent selection—AgentISA includes a capability assessment subsystem, implemented as `CapabilityProfile` in `agentflow.core.isa.capability`.

Definition 3.9 (Capability Profile): A capability profile of an agent a maps each capability domain D (the shipped enum carries eight domains: reasoning, memory, generation, optimization, specialization, coordination, security, robustness) to a score–weight pair, $\mathrm{Cap}(a) = \{(\mathrm{dom}_k, s_k, w_k) \mid \mathrm{dom}_k \in D\}$:

where each domain carries a score $s_k \in [0, 1]$ and a weight $w_k \geq 0$ reflecting the domain's importance in the current deployment context.

Implementation status. All three layers ship: the aggregation and selection mechanisms (`CapabilityProfile`, `get_overall_score`, and the coordinator's threshold selection) and—replacing the former stub—the instruction-level metric extraction itself. The seventeen `_measure_*` functions of `agentflow.core.isa.capability` extract signals statically from `FormalInstruction` declared metadata: category labels, dependency edges, parameter schemas, pre/postconditions, and declared outputs; working-memory complexity as the largest declared input arity (saturating at four); and resource economy as the inverse of the declared footprint. Extraction is deterministic, side-effect-free, and conservative: absent declarations score zero. The mechanics are pinned by unit tests (`tests/core/isa/test_capability_metrics.py`, `test_capability_pipeline.py`), including the double-weighting guard. One wiring gap remains, disclosed: `assess_capabilities` still has no production call site—the coordinator's selection path receives injected `DomainScore` values (Section 5.6's tests inject them directly).

Assessment proceeds in three steps:

1. **Domain metric computation.** For each domain, `assess_capabilities` computes a set of `CapabilityMetric` values from the agent's instruction set by static extraction, reading declared metadata only: no execution, no live probing.
2. **Domain aggregation.** Metrics are aggregated into a per-domain `DomainScore` as a confidence-weighted mean, matching the shipped `_compute_weighted_score`: $s_k = \frac{\sum_{m \in M_k} v_m \cdot \mathrm{conf}_m}{\sum_{m \in M_k} \mathrm{conf}_m}$, where M_k is the metric set of domain k, v_m the metric value, and conf_m its confidence; an empty metric set scores 0, and zero total confidence degenerates to the unweighted mean. The domain weight w_k is deliberately absent here—it enters only once, at the top level (an earlier double-counting form was removed before any reported figure; artifact notes, supplementary F.8).
3. **Overall score.** `get_overall_score` computes the weighted mean across assessed domains, $S(a) = \frac{\sum_k w_k s_k}{\sum_k w_k}$, falling back to the unweighted mean when all weights are zero.

Three properties of this design deserve emphasis. First, it is **static rather than live-probing** (runtime statistics and cross-instruction dataflow-consistency checks remain outside its scope). Second, it is **compositional**: fusing or eliminating instructions during compilation (Section 3.4) changes the profile predictably, and a production deployment would re-assess after optimization (the shipped coordinator does not yet do so). Third, it directly serves coordination: the agent-selection predicate in Section 5.6 chooses the highest-scoring agent above a capability threshold for each task, subject to the security policies of the `SecurityMonitor`—whose exercised selection path still receives injected scores, as disclosed above.

The scope of this subsystem is limited: `CapabilityProfile` assesses *declared instruction-level capability*, which correlates with—but does not measure—end-to-end task performance. Cross-model behavioral comparison at benchmark level is the domain of evaluation suites such as BFCL [17] and AgentBench [29]; the two views are complementary.

4 Implementation

This section describes the implementation status of Section 3's components, with emphasis on the compiler infrastructure and optimization techniques of Section 3.4.

4.1 Compiler Infrastructure

4.1.1 Instruction Selection Implementation. We implement the tree-pattern matching approach of Section 3.4.3 (Definition 3.8) as dynamic programming over the IR tree, in the classical manner of bottom-up instruction selection [33]. The MATCH-TREE selector ships in `agentflow.core.isa.instruction_selector` as an *opt-in strategy* (`selection_strategy="dp"`, engaging only when the input carries an `ir_tree`; a catalogue instruction declares a multi-node pattern via `params["spanned_child_ops"]`), following the Appendix D.1 recurrence exactly and exposing the DP table and root-first covering of Theorem 4.1. The *default* (`selection_strategy="heuristic"`) remains the name-overlap heuristic matcher, unchanged—so every benchmark number in Section 5 is produced under the default and none depends on tree-pattern selection.

Theorem 4.1 (Selection Optimality, shipped opt-in selector): Given a set of context-local patterns P and an IR tree T, the MATCH-TREE algorithm computes the minimal covering cost; the covering itself is read off the `selected` table afterwards, by walking from the root and skipping subtrees spanned by the chosen pattern.

Proof sketch: by induction on tree height; the recurrence sums only over children not covered by the candidate pattern, so minima compose (full argument, with the context-locality side condition, in Appendix D.1). This theorem is a property of the shipped opt-in selector, not of the default heuristic; no optimality claim is made for the default path.

4.1.2 Pipeline Scheduling Implementation. We *specify* the pipeline scheduler following the instruction-scheduling formulation of Section 3.4.3 (Deps and resource capacity) and the correctness conditions of Theorem 3.5 (Section 3.4.2), with hazard detection and resource-constrained issue in the style of classical pipeline scheduling [34]. **This cycle-accurate scheduler is design-level**: the executed runtime path is the sequential and parallel-grouped dispatch produced by the compiler's grouping pass (Section 4.1.3); no benchmark in Section 5 exercises a cycle-accurate schedule. The specified scheduler is listed in Appendix D.2.

Schedule verification checks the dependency-ordering condition (S2) of the formal model (Theorem 3.5, Appendix C.5), with the

edge convention of Section 3.4.3—$(i_1, i_2) \in$ Deps means i_1 must finish before i_2 starts. The check as written, $\forall (i_1, i_2) \in \text{Deps} : \text{slot}(i_2) > \text{slot}(i_1)$, is equivalent to $\textit{start}(i_2) \geq \textit{end}(i_1)$ only under the **unit-slot model** the shipped scheduler assumes—one slot per cycle, every instruction occupying exactly one slot. Admitting multi-cycle latencies would require an explicit *start*/*end* comparison (or interval slots); that extension remains the specified component of Section 4.1.2. S1 (exactly-once dispatch) and S4 (in-order commit) hold by construction of the scheduling loop, and S3 (group non-interference) is enforced by the grouping check.

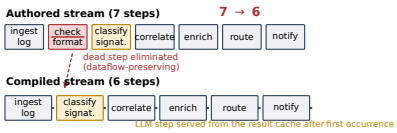


Figure 2: The compiler/hot-path pipeline on the Section 5.7.1 triage task. The authored seven-step stream compiles to six steps (dataflow-preserving dead-step elimination); the LLM-type step is served from the result cache after its first occurrence.

4.1.3 Static Optimization Implementation. We implement the optimization passes defined in Section 3.4.4. The passes that fire in the benchmarks are fusion (the instruction-combining pass of Section 3.4.4, listed in Appendix D.10 of the supplementary material), dead-code elimination (the shipped two-level design of Section 3.4.4), and parallel grouping (described with its measurement in Section 5.3). The shipped rule set registers no peephole patterns by default, so every peephole obligation below is dormant in the shipped configuration but stated for completeness (one shipped write path can populate it: approved mining-to-optimization proposals, Section 3.3.1, install name-level rules through the audited gate):

1. **Peephole Optimization** (shipped class with an empty pattern table; listing in Appendix D.8):

Each peephole rule carries a correctness *obligation*, discharged as part of rule definition rather than by the running compiler: every pattern must satisfy $\text{matches}(p, w) \implies \text{sem}(\text{replace}(p, w)) \equiv \text{sem}(w)$ for all instruction windows w.

As stated above, the shipped rule set is empty in the default configuration, so the equivalence checker is dormant; Section 4.3.1 specifies the symbolic-execution checker a populated rule set would require (the promotion-gate rules are name-level rewrites, not the textbook patterns below).

2. **Instruction Combining**: the shipped pairing pass groups adjacent compatible instructions and verifies each merged group; its full listing is in Appendix D.10 of the supplementary material, and in the source module `agentflow.core.isa.instruction_optimization`.

4.2 Runtime System

4.2.1 Dynamic Pipeline Control. Design of the dynamic pipeline controller (**specified, not implemented**: the shipped dynamic machinery is the hot-path detector (two modes; Section 4.2.2) and the trace re-optimization of Section 3.4.4, measured in Sections 5.4 and 5.8; a skeleton module exists but is not wired into the runtime, Appendix D.3). Pipeline performance is modeled with the classical pipeline bound, $\text{Throughput} = 1/\max_i \text{service_time}(\text{stage}_i)$—a model of the specified controller; nothing in Section 5 measures it.

4.2.2 Dynamic Optimization Implementation. The *default* mechanism is the counter/threshold detector described in Section 3.4.4: a monotone count per exact `path_id`, crossed against a configurable threshold with a last-recorded-success-rate gate. What the benchmarks measure (Sections 5.4 and 5.8) is this default. The Appendix D.4 detector—exponential moving average, frequency normalized by recent execution share—also ships as the opt-in `detection_mode="ema"` (details in Appendix D.4). It addresses trace diversity at the trigger level only: the detector remains exact-path-keyed (Section 3.4.4's residual still applies), and the share denominator is the LRU-tracked set, so a tail of one-shot paths can keep a later burst below threshold until the cap evicts them. Block-granular trace formation below remains specified, not implemented:

1. **Hot Path Detection** (shipped opt-in; listing in Appendix D.4):

The decision rule of the pseudocode is the threshold predicate IS-PATH-HOT (frequency above θ, stability above the stability threshold); it computes no score. A *ranking variant*—specified, not part of the decision rule—orders candidate paths by a heuristic β-weighted blend of the already-normalized frequency and recent stability, not a conditional probability.

2. **Trace Formation** (specified; listing and validity definition in Appendix D.5): trace validity is the conjunction of three properties—entry dominated by the loop header, exit post-dominating the tail, no side exit except the tail—checked by VERIFY-TRACE-PROPERTIES.

4.3 Optimization Verification

4.3.1 Static Analysis. We *specify* verification of the static optimizations from Section 3.4.4 (a symbolic-execution equivalence checker over transformation path pairs). As stated in Section 4.1.3, the shipped peephole table is empty and this obligation is dormant; the soundness theorems (Theorems 3.4–3.6) are conditional on discharging it when rules are introduced (Appendix C.7). The verification follows the formal model, $\text{Equiv}(p_1, p_2) \iff \forall \sigma \in \Sigma : [\![p_1]\!]\sigma = [\![p_2]\!]\sigma$; the specified checker is listed in Appendix D.6.

4.3.2 Dynamic Analysis. The runtime *monitors* optimizations through a single mechanism (no result-correctness oracle is defined at runtime—correctness of LLM-instruction results is not decidable in general and is delegated to the authoring-time verification of Section 3.3.2; the monitor below is a specified, not shipped, component):

1. **Performance Monitoring** (specified; listing in Appendix D.7).

5 Empirical Evaluation

We evaluate AgentISA with a reproducible microbenchmark suite (`examples/benchmarks/isa_microbench.py`) and a multi-agent functional test suite, under the scope stated at the end of Section 1.3.

5.1 Methodology

5.1.1 Instrumentation: Mock LLM Backend. All experiments route model inference through a `MockLLMBackend` that counts each `infer` call as one simulated LLM API round trip and charges a fixed 80 ms simulated latency per call to a simulated clock. It never sleeps and never contacts a live model, so every "API call" is observable, deterministic given a seed, and reproducible without network access or inference spend. Everything *around* the backend is real: the cache, compiler, and optimizer are the production `agentflow.core.isa` implementations (`InstructionCache`, `InstructionCompiler`, `OptimizationManager`), exercised through the same code paths a deployment uses.

5.1.2 Workloads. Workloads are synthetic and seed-controlled:

- **Caching**: 2,000 tasks from a discrete distribution over 50 instruction templates × 20 parameter values, popularity Zipf-distributed with exponent $s \in \{0.5, 1.0, 1.5\}$. Three arms share instrumentation (Section 5.2): no cache, the `InstructionCache`, and a naive exact-response dict keyed on the request payload.
- **Static compilation**: random sequences of length 50, 200, and 1,000 from `synthetic_formal_sequence` (`examples/benchmarks/isa_microbench.py`; short enough to audit line by line). Each of the first $n-1$ slots is drawn independently: with probability 0.25 a *dead* instruction (side-effect-free, **no declared outputs, hence no consumers**—removable by construction, planted not inferred); with probability 0.55 a *pure* instruction (side-effect-free, one declared output v_i, consumed **only by the final sink**, fusable next to a same-type pure neighbor); otherwise *side-effectful* (always live, never fused or removed). The n-th slot is a side-effectful *final sink* depending on exactly the set of all pure outputs, so every pure instruction has a downstream consumer and no pure output is deleted as dead. The attributes are disjoint and jointly exhaustive per slot; the passes find the planted DCE/fusion targets, so the measured reductions are properties of this workload family, not of arbitrary agent programs.
- **Dynamic optimization**: instruction traces of length 100 recorded for up to 200 iterations per seed, hot-path detection active (threshold 50 recordings). Each trace uses the static arm's slot-draw shape at a different mix—dead 0.30, pure-with-output 0.50, side-effectful 0.20—plus a side-effectful final sink depending on all pure outputs (seed base 3,000; recorded in the results file's config). With post-recording counting and a strictly-exceeding trigger, the optimizer first fires on recording 51 (the run's 0-based iteration 50), so each seed contributes exactly 51 recordings (50 counted plus the trigger).
- **Ablations**: two arms. The primary arm is *paired*: it recompiles the static-compilation sequences (identical seed base 2,000) with fusion and dead-code-elimination toggled individually and jointly—a same-input decomposition of the 34.2% headline. The secondary arm is an *independent* draw, same distribution but seed base 4,000, a fresh-sequence replication (all-passes total 35.0%, within one standard deviation of the headline).

5.1.3 Protocol and Statistics. Each synthetic-workload configuration (Sections 5.2–5.5) is run with 5 random seeds; we report mean ± sample standard deviation. These are **descriptive statistics only**: no hypothesis-test claims (no p-values, no significance tests) anywhere in this paper. Five seeds characterize variability across the tested workload instances, not statistical significance or population-level generality. Section 5.2's naive response-cache baseline shares the mock-backend instrumentation; a formal *inferential* comparison against full external systems remains out of scope, and no such claim is made.

5.1.4 Metric Terminology. Several distinct quantities must not be conflated; each is defined once here:

- **Effective-instruction reduction**—fewer *effective* instructions in the compiled program (Sections 5.3/5.5); an instruction-stream property, not execution time.
- **Dispatch-count reduction**—fewer dispatch operations at runtime, including parallel grouping's folding (Section 5.3); a runtime count.
- **Trace shrinkage**—shorter traces after hot-path reoptimization (Section 5.4); the optimized trace's executed-state identity is verified separately there.
- **Cache hit rate / call reduction**—fraction of calls served from the result cache; here the call-reduction ratio equals the hit rate by construction (Section 5.2).
- **Simulated latency saving**—avoided-call counts × the mock's fixed 80 ms (Section 5.2); derived, not measured.
- **Measured latency/token saving**—wall-clock time and token counts against the real gateway (Sections 5.7–5.7.1); measured.

For context preservation (Theorem 3.3): *extraction-time preservation* is the extraction step's copy invariance; *runtime context validation* is the monitor's pre/post-dispatch constraint checking—different properties, different coverage (Section 3.2.2 discloses the deletion blind spot).

5.2 Experiment 1: Result Caching

Research question. How many simulated LLM API calls does AgentISA's result caching eliminate, as a function of workload skew?

We execute 2,000 tasks per seed, with caching disabled and with the `InstructionCache` enabled, measuring simulated API calls at the backend; the reduction ratio and hit rate coincide by construction (every avoided call is exactly one cache hit).

Cache-key contract. The shipped result cache keys an entry on the (instruction, params) pair—realized as a params-only SHA-256 scoped per instruction instance—deliberately *not* on model identity/version, prompt-template version, tool schema, decoding parameters, or ambient context. Soundness requires

Zipf s	No-cache calls	Cached calls	Reduction
0.5	2,000 ± 0	810.4 ± 5.7	59.5% ± 0.3%
1.0	2,000 ± 0	642 ± 3.8	67.9% ± 0.2%
1.5	2,000 ± 0	408 ± 9.4	79.6% ± 0.5%

those factors held fixed, which every benchmark here guarantees by construction (single alias, fixed templates and decoding, fresh cache per arm); Section 5.7.1's classifier-keyed step hashes exactly the input dictionary its prompt is built from, under the same discipline. A deployment changing any of these factors must invalidate or namespace the cache. The production cache isolates by TTL, LRU capacity, and never-cached failures, and an opt-in strict namespace ships for this drift (`CacheKeyNamespace` in `agentflow.core.isa.instruction_cache`): the key extends (instruction, params) with model alias, template version, and a canonicalized decoding hash, with isolation-not-eviction semantics; the default key stays the params-only SHA-256, byte-identical to the published hit statistics. *Automatic* drift detection remains absent, and no number in this paper runs under the strict key.

Naive response-cache baseline. A third arm runs an app-level *response* cache keyed on the exact request payload string—what an exact-match cache in front of the backend keys on—implemented as a plain dict with no AgentISA machinery. Across the three skews it issues exactly the same call counts as the table's cached arm: **parity with the `InstructionCache` to the digit**, because the (instruction, params) key is already canonical on this workload. Parity is the finding: on call reduction the instruction-level cache adds nothing over exact-response caching—its value is reducing the program before any request is issued (Sections 5.3 and 5.5), which a response cache cannot express. Parity is also a property of this workload, not the mechanism: under random param-insertion order the instruction key hits 95.0% versus an insertion-order response key's 7.3%—a gap that vanishes (95.0%, 0.0 points) under a response key canonicalizing param order, so the instruction key's advantage there is canonicalization alone; under template churn the default key false-hits 48.3% of calls, which the strict namespace key (90.4% hit, zero false hits) prevents (supplementary I.4). Semantic/embedding caching remains out of scope (Section 5.9).

(data: `isa_microbench_results.json`, cache experiment)

Interpretation. The reduction increases across the three tested skews, 59.5% at $s = 0.5$ to 79.6% at $s = 1.5$: the savings' upper bound is the probability mass of repeated tasks, which the Zipf exponent directly controls. In simulated latency the savings are the avoided-call counts × 80 ms ($s = 0.5/1.0/1.5$: 95,168 ± 455 / 108,640 ± 305 / 127,360 ± 750 ms)—derived, not separately measured.

Two caveats apply. First, these are *simulated* calls against a mock backend: the reduction is exact for this workload family, but real deployments face cache-invalidation, prompt-context, and embedding-variability effects the mock abstracts away. Second, the absolute number depends on reuse structure; workloads without repetition would see zero benefit. The experiment establishes the *mechanism and its skew sensitivity*, not a universal constant.

5.3 Experiment 2: Static Compilation

Research question. How much does the compiler reduce instruction sequences, and what do the different optimization levels contribute?

We compile random sequences at four optimization levels and measure two quantities, kept explicit: **effective instruction count**—instructions remaining after fusion and DCE, a program-size measure—and **dispatch count**—dispatches the runtime issues; parallel grouping folds several independent instructions into one dispatch without changing the effective count.

Interpretation (full table: supplementary I.3). Three observations:

1. **Effective instruction count reduction is level-stable**: BASIC, ADVANCED, and AGGRESSIVE produce identical reductions (34.2% ± 1.8% at length 200). BASIC's fusion and DCE already capture the available elimination gains; higher levels add *scheduling* improvements, not elimination—ADVANCED and AGGRESSIVE coincide here, differing only on streams carrying combinable patterns.
2. **Dispatch count drops further with parallel grouping**: at length 200, dispatch falls from 131.6 (BASIC) to 74.0 (ADVANCED) with effective-instruction reduction unchanged—adjacent side-effect-free instructions with disjoint dependencies and outputs grouped into shared dispatches. The 74.0 figure is a scheduling reduction, not a further program-size reduction (an earlier pass folded more but on unproven non-interference—history in supplementary F.8).
3. **Gains are stable across sequence lengths**: 32.8%, 34.2%, 35.4% at lengths 50, 200, 1,000 respectively, tightest variance at length 1,000 (± 1.6%); length does not materially change the opportunity structure.
4. **A non-planted-family control bounds the headline.** Over Section 5.10's corpus (100 workflows, 792 steps) the same compiler reduces 19.2%, all dead-guard elimination: pure transforms there form dataflow chains, each of the 106 fusions absorbs a dead guard, no live pair passes the disjointness gate—the fusion share of 34.2% belongs to the planted linear family (supplementary I.4).

Effective-instruction reduction is again a *program-size* metric: it translates to simulated API-call savings only where eliminated or fused instructions carry model inference; the caching experiment measures the complementary reuse effect.

5.4 Experiment 3: Dynamic Hot-Path Optimization

Research question. Does the dynamic optimizer detect and specialize repeatedly executed traces?

We record an instruction trace of length 100 for up to 200 iterations per seed with hot-path detection active (Section 5.1.2). Once a path is flagged hot, the runtime forms an optimized trace; **this experiment measures that trace's length and re-executes it**, the replay check verifying exact reproduction of the original trace's final

state (task-level artifact preservation on realistic streams: Section 5.8).

Across seeds, hot-path optimization shrinks the trace by **65.8% ± 4.6%**; the seed-to-seed spread (59.0%–70.0%) reflects which subsequences each trace makes hot. As with the static experiments, this is trace-length reduction on synthetic traces—workload-controlled evidence that the loop functions, not a wall-clock speedup claim.

Replay verification. Each seed's optimized trace *is* re-executed—every instruction reads its declared dependencies from a threaded state dict and returns deterministic bindings for its declared outputs (side-effectful instructions leave per-id markers)—and its final state is compared with the original's, executed the same way. The states are identical in all five seeds (5/5; recorded per seed as `replay_state_identical` in the results file), so the 65.8% shrinkage carries an executed-equality check at trace level—on the deterministic mock, not against a live model; this is the same style of check as Section 5.8's task-level byte-identity, applied to the synthetic traces. Live-model behavior remains untested—no mock experiment can address it.

5.5 Ablation: Per-pass Contribution

To attribute the static gains we run the two ablations of Section 5.1.2. The primary one is **paired**: it compiles the *same* sequences as Section 5.3 (identical seed base 2,000 and generator) with fusion and DCE toggled individually and jointly, so per-configuration differences are same-input paired differences—an exact decomposition of the headline, not a replication:

The paired per-pass decomposition (table: supplementary I.3) gives fusion+DCE = 34.2% ± 1.8%—the headline itself, confirming that these two passes account for the entire static gain at length 200. The **paired interaction**, computed per seed as (fusion+DCE − NONE) − (fusion − NONE) − (DCE − NONE)—34.2 − 21.6 − 24.5 = −11.9 points—is −11.9% ± 2.0% (per-seed −14.5/−9.5/−13.0/−12.0/−10.5 points): the passes overlap—dead slots break fusion chains and fused pairs absorb neighbors DCE would otherwise have removed individually—so the sum of the single-pass gains (46.1 points) overstates the joint effect by 11.9 points.

The secondary ablation is an **independent-draw replication** (seed base 4,000, same distribution): fusion alone 21.9% ± 3.5%, DCE alone 24.1% ± 3.3%, fusion+DCE 35.0% ± 3.7%—the paired structure reproduces on fresh sequences. Because dead instructions are planted at rate 0.25, DCE's per-pass share is near-tautological for this family—it measures that the pass finds the planted dead code, precisely its function.

5.6 Multi-Agent Coordination: Functional Validation

The multi-agent layer—`AgentCoordinator`, `SecurityMonitor`, and the collaborative instruction set—is validated functionally by 43 tests, all passing—the coordination-and-security subset of the repository's full suite (1,358 passing, 14 skipped at run time), which also covers the ISA core, compiler, and optimizer units of Sections 5.2–5.5. This is **qualitative functional validation**: the tests verify the coordination behaviors exist and are correct, not that they are fast; no performance number should be inferred here.

The coordination flow implemented by `AgentCoordinator` is listed as pseudocode in Appendix D.10 of the supplementary material. What the 43 tests verify, grouped by subsystem:

1. **Agent registration and role policies**: registration creates policies; permissions and security requirements differ by role; trust levels are bounded and reward declared specializations.
2. **Task coordination**: coordination produces a solution or records failure; missing authentication/authorization and no-suitable-agents raise; selection respects the capability threshold, and the selected coordinator is the highest-scoring agent.
3. **Execution planning**: stage structure with coordination rules; every stage has differentiated fallback keys and a fallback executes on stage failure; validators are reasoning-heavy non-coordinator agents.
4. **SecurityMonitor**: valid tasks pass, missing authentication/authorization fails, violations are timestamped, declared data sensitivity is enforced (sensitive categories infer a high sensitivity level, benign ones pass at medium), resource access is open by default and restrictable by configuration.
5. **Collaborative instructions**: barriers wait for the declared participant count and time out on a missing peer; resource allocation rejects over-allocation and serializes concurrent requests; the workflow state machine accepts valid actions and rejects invalid ones and double starts; a research-team scenario runs end to end.

These tests exercise the same production modules used elsewhere and are fully deterministic (no LLM calls). They establish correctness of the coordination machinery under the tested conditions; what they do not establish is scalability under contention, crash recovery, or adversarial inputs beyond the policy checks above—all future work (Section 6).

5.7 Live-Gateway Conformance Check of Result Caching

Research question. Does the caching mechanism behave identically when the "model" is a real LLM behind a live gateway, and how much *measured* wall-clock time and token usage does it save?

Setup. We replace the mock backend with a real model behind a local Anthropic-protocol gateway (TLS with an internal CA; alias `claude-glm-5.3-flash`, a hosted reasoning-capable GLM model; `max_tokens=64`; sequential httpx, 90 s timeout, three bounded retries—none occurred). Every other component is the production code path: the same `InstructionCache` protocol, instruction objects, and workload generator, with Section 5.2's instruction pool (50 templates) and parameter space (20 values).

Scale. With each round trip costing ~2.2 s, scale drops from 2,000 tasks × 5 seeds to **200 tasks × 1 seed** per Zipf level (s in {0.5, 1.0,

1.5}): 1,040 measured round trips (plus one preflight) in 39.4 minutes. The seed pins the task sequence exactly, so the workload is the one the expectation below is computed from.

An exactness property. A cache hit depends only on the (instruction, params) pairs seen so far—not on what the model returns. In general an LRU-with-TTL hit also depends on wall-clock time and capacity; under this benchmark none of that can bind: `max_size` = 100,000 (at most 200 distinct keys per arm), `ttl` = 3,600 s (every cache-enabled arm completes in under seven minutes; the no-cache arms run to 7.9 minutes and never consult a cache), a fresh cache per arm, failed calls never cached. Hit/miss decisions are therefore a pure function of the (instruction, params) sequence, *computable without any backend*; if the mechanism is correct the real run must reproduce that sequence exactly—and at the recorded granularity (per-arm hit rates and call counts) it does:

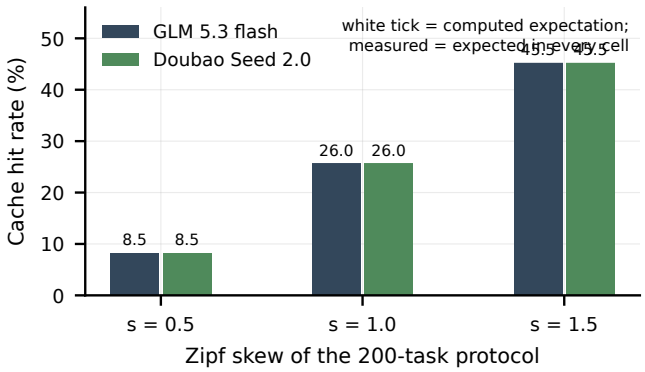


Figure 3: Live-gateway cache conformance across model aliases. Bars are measured hit rates under the identical protocol (200 sequential requests, one seed, three skews per model); the white tick is the backend-free computed expectation—matched exactly in every (alias, skew) cell; call, wall-clock, and token figures: supplementary I.3.

Per-call latency was stable across arms (p50 2.13–2.21 s, p95 3.06–3.54 s). Token savings are gateway-reported `usage` differences: every avoided call saves its input tokens plus the tokens the reasoning model generates before responding (output tokens were the binding constraint of `max_tokens=64` in every call).

Interpretation. First, the mechanism transfers unchanged: observed hit rates match the computed expectation exactly at every skew level, so Section 5.2's synthetic results measure the cache, not a mock artifact. Second, avoided calls convert into measured time and tokens—14.1%, 26.6%, and 45.8% of the no-cache arm's wall clock at s = 0.5, 1.0, 1.5—tracking but not identical to the call reduction (wall and token figures are measurements). At N = 200, one seed, the wall-clock ratio is sampling-noise-dominated (the 17 avoided calls average 1.7 × the arm mean); the token ratio is the steadier read. Third, the absolute hit rates (8.5–45.5%) trail Section 5.2's (59.5–79.6%): repetition accumulates with task count—the backend-free computation at N = 2,000 reproduces the Section 5.2 figures (the gateway run stays at N = 200).

Multi-model rerun. The identical protocol re-run on a second production model (`claude-doubao-seed-2.0-code`, replies completing below the ceiling) reproduces the expectation exactly in all three cells—8.5%, 26.0%, 45.5%; calls 200 → 183/148/109; p50 3.6–4.0 s (table: supplementary I.3). The exactness property is model-independent by construction, so this rerun is a conformance check of the implementation on a second platform, not new evidence about model variation.

Two-seed rerun. A reduced-scale rerun (100 tasks × 2 seeds per Zipf level) reproduced the computed expectation exactly in all six (Zipf, seed) cells (hit rates, quota record: supplementary I.3).

Scope of the claim. This experiment is a conformance check of the caching mechanism's implementation—one gateway, two model aliases (one seed per alias at N = 200, plus the two-seed reduced-scale rerun: supplementary I.3), sequential requests, no streaming—recording per-arm aggregate hit rates and call counts; per-position identity follows from the deterministic cache decisions above but is not separately logged, and nothing here is a throughput, availability, or dollar-cost claim. Sections 5.3–5.5 are structural, never dispatching to the backend; Section 5.7.1 extends the real-gateway validation to them via a cache-mediated protocol.

5.7.1 Live-Gateway Identity Check of Compilation and Hot-Path Optimization. **Research question.** Do static compilation and closed-loop hot-path adoption preserve the observable semantics of an instruction stream when the stream's LLM step is served by a real model—and what do they measurably save on a live gateway?

Setup, arms, scale. A dedicated harness (`examples/paper_real_llm_compiler_test.py`; tests in `tests/system/test_real_llm_compiler_test.py`) reuses Section 5.7's gateway configuration (alias `claude-glm-5.3-flash`, sequential httpx, bounded retries) but raises `max_tokens` to 512 (Section 5.7 used 64) so completions dominate the priced comparison; it runs identically online or offline against a deterministic same-protocol mock. The task is Section 5.8's service-log triage shape: a seven-instruction stream with one cacheable LLM-type step (`classify_signature`, one model call per distinct incident-signature batch) and the same planted dead guard (`check_format`), over 50 realizations × 1 seed × Zipf s = 1.0 from a fixed universe of 40 task instances—22 distinct signature batches, 23 gateway round trips in 4.4 minutes—bounded validation, not Section 5.7's 200 × 3-skew sweep. Arm (a) is the unoptimized authored stream, run first, cold, performing every live round trip; arm (b) drives the *same* stream through the production machinery—`optimize_hot_path` adoption plus static `InstructionCompiler` at NONE vs AGGRESSIVE with the dataflow-preservation assertion—then executes the same draws against arm (a)'s warmed cache. The classify step's cache key is the sha256 of exactly the input dictionary its prompt is built from, so arm (b) issuing *zero* new gateway calls proves it computed identical keys—hence identical prompts and consumed values—and its artifacts must come out byte-for-byte equal.

The +1 preflight is the token-ceiling probe.

Run	Alias	max_tokens	Skews	Requests issued
Cache conformance (§5.7)	claude-glm-5.3-flash	64	0.5/1.0/1.5	600 (200/skew; hits client-side)
Identity check (§5.7.1)	claude-glm-5.3-flash	512	1.0	23 (1 preflight + 22 live)

Results. All checks pass. Arm (a) made exactly the 22 first-occurrence calls the backend-free computation predicts and served the other 28 realizations from the result cache (hit rate 56.0%), consuming 4,027 prompt and 11,264 gateway-reported completion tokens in 260.2 s of measured call latency. Arm (b)—compiled, hot-path-adopted—made zero new gateway calls and reproduced every artifact byte-for-byte (the check is sensitive to any transformation that perturbs a cache key or an emitted artifact byte; divergence confined to non-keyed, non-emitted runtime state would escape it). The hot path was adopted at realization 7 (threshold 5; adoption lands two realizations past it because the counter must strictly exceed it and recording follows execution, as in Section 5.8): realizations 1–6 executed the full seven-instruction stream, realizations 7–50 the shrunken six-instruction form with `check_format` absorbed by fusion—306 instruction executions against 350 in arm (a), a 12.6% executed-instruction reduction attributable to the adopted trace (per-stream shrink 14.3%, 7 → 6; the static compiler alone reproduces the 7 → 6 shrink at AGGRESSIVE, so the two mechanisms' contributions are not separated), reported separately from the cache's call savings. Static compilation of the same stream: NONE 7 → AGGRESSIVE 6 effective instructions, dataflow preserved.

Identity rerun with live-model values. Re-running the harness on `claude-doubao-seed-2.0-code` at `max_tokens` = 1,500 changes the evidential class: all 104 live calls across the three Zipf cells completed below the ceiling (max completion 142 tokens, 0 bound) and replied in the requested reply format, so the artifacts' severity values are the live model's own classifications, not the fallback. Every mechanism check re-passed—expected call counts, arm (b) zero new calls, byte-identical artifacts, hot-path adoption, the 7 → 6 shrink—and a third model (`claude-minimax-m3`, reasoning-style) passes every check while binding at the ceiling in 97% of calls at `max_tokens` = 512 and 65% at 1,500: ceiling-binding is a model-alias property, not a harness artifact (census: supplementary I.3).

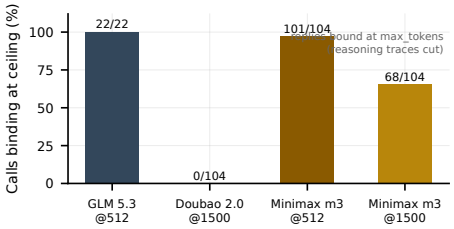


Figure 4: Ceiling-binding census across model aliases and budgets. Calls binding at `max_tokens` per alias and budget, from the identity runs' raw records: GLM 5.3 flash binds in 22/22 calls at the compiler budget (512), Minimax m3 in 101/104 at 512 and 68/104 at 1,500, Doubao Seed 2.0 in all 104 calls below the ceiling at 1,500.

A priced comparison (summary; protocol and table in Appendix G of the supplementary material). Costs are MODELED on measured tokens, not observed: the overlay prices this run under no optimization, AgentISA result caching (first occurrences only), and a modeled provider prompt cache (repeat prompts bill as 0.1x cache reads, mirroring Anthropic's published discount; completions at full price); two simplifications favor the provider-cache baseline, and list prices are parameterized stand-ins—no dollar-savings claim is made. On this run's token mix the result cache cuts modeled cost by 56.0% while the prompt cache cuts only 3.3% (the reasoning model's completions dominate); on the mock at paper scale the corresponding figures are 80.1%, 81.6%, and 85.1% against 28.9–31.4%—the ranking of the two caching strategies is a function of the input/output token mix, not a constant. GPTCache is explicitly out of scope; no head-to-head is claimed.

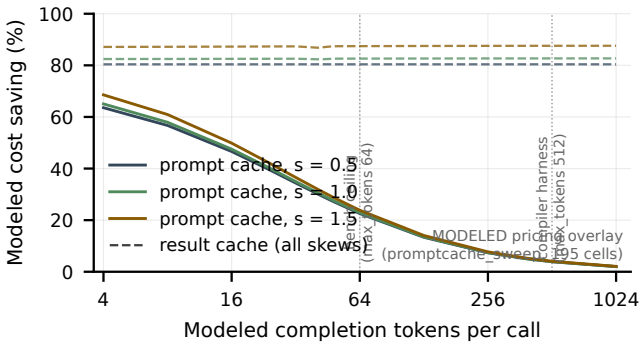


Figure 5: Token-mix sweep of the priced comparison (MODELED overlay, 195 cells). The provider prompt cache's saving collapses as completions lengthen (63.6% at L4 to 2.0% at L1024, s = 0.5) while the result cache holds 80-88%: the ranking is a function of the token mix, not a constant (protocol: supplementary I.3).

What this does and does not show. The byte-identity property is cache-mediated by design: the values entered the artifacts once through arm (a)—fallback-derived on this alias, live-model classifications in the doubao rerun—and arm (b) is proven to compute the same keys and consume the same values; this validates the stream transformation, not model determinism, which is not guaranteed at temperature 0 and is not claimed. On this alias the reasoning model's replies bind at `max_tokens` in every call (as at `max_tokens=64` in Section 5.7), so the classify step's severity values come from its deterministic fallback; what is validated against the live model is the mechanism chain (key computation, cache decisions, call and token accounting, artifact identity), not classification quality.

What the real-gateway experiments establish, in three layers kept separate: (1) *gateway integration correctness* (requests reach the gateway, the cache governs calls, call/token statistics recorded faithfully) and (2) *workflow semantic preservation* (the optimized stream computes the same cache keys and consumes the same cached values, yielding byte-identical artifacts—as a cache-mediated identity validating the stream transformation, not model determinism): **established** by Sections 5.7 and 5.7.1. (3) *Real-model task quality*: **measured only by a follow-up rubric diagnostic**—a 3,072-token rerun of the same 22 signature batches parsed 144 of 176 claims, with final classifications matching the planted keyword-rubric gold on 45.5% of signatures (model claims 51.4%) against 32.4% for the deterministic fallback rule; 4 of 22 batches still bind at the ceiling—a rubric diagnostic, not a deployment-quality claim. A second quality run on a completing model (`claude-deepseek-v4-flash`, 36 distinct signature batches at the paper preset, `max_tokens` 1,500) parses 288/288 claims with 0 ceiling bindings: final classifications match gold on 52.1% of signatures against 31.9% for the fallback rule (per-class agreement 67.6/39.0/47.8/54.4% on low/medium/high/critical)—model-over-rule holds on both families (protocol: supplementary I.3).

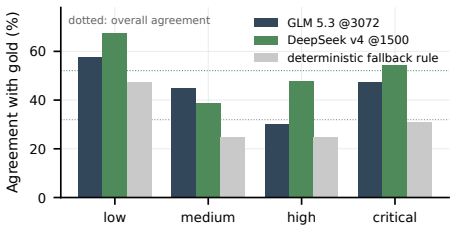


Figure 6: Per-class agreement with the planted rubric gold. Both live models (GLM @3072, 176 signatures; DeepSeek @1500, 288/288 parsed, zero fallback fills) against the fallback rule recomputed per class from the DeepSeek run's raw records; dotted lines mark overall agreement.

5.8 Controlled Four-Task System-Level Characterization

The mechanism experiments above measure sequences; Appendix H reports the controlled four-task system-level characterization in full—triage, audit, migration, and support shapes of seven to nine instructions, twelve realizations each, driven through the shipped engine's own `record_execution` and `optimize_hot_path` API rather than a reimplementation of the passes. All four tasks calibrate Section 5.4's synthetic headline downward to 12.5–25.0% per-task trace shrink, with the eliminated set empty everywhere (the reduction is fusion, with a second-order component from guard removal) and uniform reuse and closed-loop adoption across shapes.

5.9 Limitations and Threats to Validity

We enumerate the limitations of the evaluation explicitly:

1. **Mock backend, not a live LLM.** The mock charges a constant 80 ms per simulated call, returns deterministic responses, and covers Section 5.4's replay verification (final-state identity, 5/5 seeds) and Section 5.2's naive-cache baseline. The real-gateway experiments extend coverage to the cache's hit/miss reproduction with measured time/token savings (Section 5.7) and the compiled stream's zero new gateway calls and byte-identical artifacts (Section 5.7.1). Still not covered: multi-seed behavior beyond the two-seed rerun (Section 5.7), real deployment, caching under model nondeterminism, streaming, provider rate limits, and live-model quality beyond the Section 5.7.1 rubric diagnostics (the four-task classification steps remain rule-based stand-ins).
2. **Synthetic workloads.** Zipf-distributed synthetic tasks reasonably model skewed agent workloads—but are a model; Section 5.8 partially addresses this, and Section 5.10's generated-trace census does not (it characterizes the generator's own vocabulary). The cache-reduction figures (59.5–79.6%) are exact for these workload families and should be read as *skew-sensitivity curves*, not universal savings; uniformly novel tasks would see near-zero benefit. Section 5.8's four-shape characterization calibrates the 65.8% synthetic headline downward with its 12.5–25.0% per-task compile shrink and shows uniform reuse and closed-loop adoption—but the four tasks remain hand-constructed, not a production workload study. Section 5.12 adds recorded model trajectories: static reduction there is 0.0% and cache reuse is 41.7% across tasks but 1.7% within one, so each headline is a bound of its construction, not a workload constant.
3. **Generated corpora do not establish realism.** The census corpus is itself a generator artifact—its 12-signature vocabulary, log-linear decay, and 98.5% recurrence characterize the generator, not any production workload. The non-planted-family control (supplementary I.4) swaps planted linear sequences for the same generator's traces (19.2% vs. 34.2%)—a generated-versus-planted comparison; Section 5.12 supplies the recorded-trajectory arm.
4. **Capability correlations are small-sample consistency checks.** The domain-score correlations (Section 5.11) are computed on n = 25 declaration-injection pools (generation n = 15), with scores and metrics derived from the same injected declarations—consistency checks, not calibration. The ± 1.000 entries are step-function artifacts of declared-and-enforced gates, and no independent, live-model capability measure enters any correlation.
5. **No production end-to-end evaluation.** We have not deployed AgentISA in a production setting; the paper therefore contains no latency-under-load, throughput, availability, or dollar-cost measurements, and makes no such claims.
6. **No real cost data (tokens measured; pricing MODELED).** Sections 5.2–5.5 latencies are simulated (avoided calls × 80 ms); Section 5.7 measures wall-clock time and gateway-reported tokens; Section 5.7.1 prices measured tokens under three modes—no optimization, result caching, and a modeled provider prompt cache (cache-read discount 0.1). Prices are configurable list-price stand-ins; no dollar-savings claim is

made. Approximate (semantic/embedding) caching remains out of scope and future work.

7. **Program-size metrics, not wall-clock time.** Effective-instruction and dispatch-count reduction are structural metrics: they correlate with—but do not equal—wall-clock improvements, which depend on per-instruction cost, available parallelism, and dispatch overhead.

8. **Descriptive statistics only.** Synthetic configurations use five seeds (Section 5.1.3); the real-gateway and system-level runs are single-seed by design (Section 5.7's two-seed rerun excepted); no hypothesis tests, confidence intervals, or significance claims are made—standard deviations indicate stability across seeds, nothing more.

9. **Functional, not performance, validation of coordination.** The 43 multi-agent tests verify behavioral correctness under deterministic conditions, not throughput or scalability.

10. **No prompt-variant robustness evaluation.** Each experiment measures one authored instruction stream per configuration (Sections 5.2–5.5, 5.8). Outcomes under prompt paraphrase—semantically equivalent requests yielding different (instruction, params) keys and responses—are unmeasured; the exact-match key design forfeits reuse there by construction, but the sensitivity is not quantified.

Despite these limitations, each optimization mechanism is implemented, active, and produces the claimed structural effects on workloads designed to expose them.

5.10 Generated-Trace Vocabulary Census

This census characterizes the generator's own vocabulary; it does **not** discharge the synthetic-workload threat of Section 5.9. On 100 seeded random-task stress traces (792 steps, 12 signatures) the rank-frequency decay is log-linear with descriptive slope 0.593, and 98.5% of instances recur corpus-wide (780/792), while per-workflow repetition, 18.3%, is more informative—reuse here is cross-task vocabulary, not intra-task duplication (protocols and tables: supplementary I.1, I.3).

5.11 Mechanism Extensions

Six experiments exercise opt-in mechanisms (protocols and tables: supplementary I.2). **Capability calibration:** domain scores correlate with mock-backend behavior on declaration-injection pools: memory +0.948, coordination −0.977 (the sign is the designed coupling: more declared edges, more fusion barriers), reasoning −0.965, generation +0.790; optimization and security are step-function gate artifacts—consistency checks, not calibration—and with 2/8 domains, 9/17 metrics N/A. **DP tree-pattern selection:** covering cost $\leq$ heuristic on 20/20 IR trees (ratio 0.361; time 0.950). **EMA hot-path detection:** 3/3 rotated variants promoted (exact: 0/3; Zipf recall per seed: [0.333, 1.000, 1.000, 1.000, 1.000] (mean 0.867)), zero false promotions on one-shot diversity. **Strict security:** fail-closed denies 400 policy-gap tasks fail-open admits; 0/200 normal-task false denials. **Cache namespacing:** 100% invalidation under template/decoding drift; modeled cost no higher than the provider prompt cache in 195/195 token-mix cells. **Parameterized replay:** all 1,050 unseen-parameter replays match the oracle exactly. Chain-enriched workloads amplify BASIC→ADVANCED dispatch reduction to 79.1% (random-slots control 46.7%—paralleling Section 5.3's 43.8% random-sequence dispatch reduction (131.6 → 74.0 at length 200; the control's 46.7% is measured on its own length-200 slots); protocol and per-seed tables: supplementary I.2), AGGRESSIVE differing only in the advisory memory plan.

5.12 Do the Reductions Survive Real Traces?

Every number above comes from workloads we constructed. We therefore ran the shipped compiler and result cache on 1,960 tool-call trajectories recorded from real model runs (τ-bench; GPT-4o and Claude 3.5 Sonnet; retail and airline; 14,285 calls), converted under the conservative mapping in supplementary I.6: same-turn calls are independent, later turns consume the preceding step's result, unknown tools are effectful, and final-turn outputs are required so liveness cannot delete the answer. Static reduction is **0.0%** at BASIC and AGGRESSIVE—real traces chain their observations, so no step is provably dead and no pure pair provably independent, confirming Section 5.3's 34.2% as a property of the planted family. Reuse transfers conditionally: **1.73%** of cacheable calls hit within a task, **41.70%** across the distinct tasks of one run (retail 49.0–50.2%, airline 16.3–22.1%), and 83.4% under the benchmark's trial replay, which repeats identical tasks rather than exhibiting workload repetition. Section 5.2's benefit therefore survives on real workloads only where a cache is shared across tasks. A second, bash-only coding-agent family (supplementary I.6) also earns 0.0% statically and contains no cacheable instruction at all, though 47.2% of its commands repeat verbatim: repetition is not cacheability. Do instructions themselves sediment, so that reuse pays as experience accumulates? Yes, and Figure 7 measures it: 12–14 active templates serve **95.9%** of the 2,340 tool-use steps and 103 serve 84.9% of household manipulation, while 291 command templates serve 43% → 67% on shell coding and 16 serve 45.2% of SQL analytics. A finite action space is therefore necessary but not sufficient; parameter reuse governs the payoff (protocols: supplementary I.7, I.8).

6 Discussion

6.1 System Limitations and Challenges

1. **Instruction-set coverage**: complex reasoning patterns are not yet naturally expressible (domain-specific extensions are likely needed), and maintaining consistency of pre/post-conditions and typing rules across extensions remains an open engineering challenge.

2. **Evaluation scope**: the quantitative evidence is primarily microbenchmark-based (Sections 5.2–5.5), supplemented by real-gateway spot checks (Sections 5.7–5.7.1) and the four-task characterization (Section 5.8); production end-to-end evaluation remains required before any deployment claim (Section 5.9).

3. **Performance bottlenecks**: cache memory consumption under large instruction universes; synchronization overhead in barrier-coordinated multi-agent execution; recompilation cost

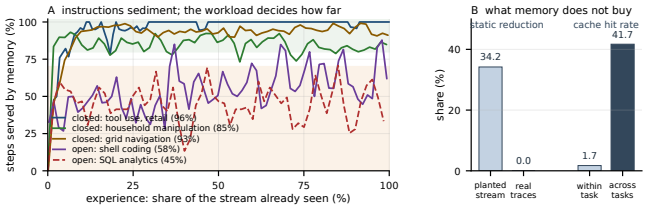


Figure 7: Instruction memory is an asset, and its payoff is workload-shaped. (A) Share of a stream's steps served by instructions mined from earlier tasks, versus experience, on five recorded task families (first trial per task; promotion at two occurrences; a step is credited only to an instruction promoted in an earlier task). Closed argument vocabularies saturate at 85–96% almost immediately; open ones climb slowly (43% → 67%) or flatten near 45%, because the verb is shared while the parameters are not. (B) What the same memory does not buy: the 34.2% static reduction belongs to the planted family, not to recorded traces (0.0%), and reuse pays only across tasks (41.7% versus 1.7% within one).

when hot-path thresholds are misconfigured; the *default* hot-path detector keys on exact sequence identity, so trace diversity never accumulates counts (the opt-in EMA detector of Section 4.2.2 is the remedy, still exact-path-keyed—its denominator can be floored by one-shot tails); state is bounded by the LRU cap (default 1,000), but recency-only eviction can drop a hot path's state, not just cold ones.

4. **Integration challenges**: adapter maintenance as provider function-calling surfaces evolve; version management of learned instructions across LLM updates.

6.2 Security and Privacy Considerations

AgentISA's implemented security layer is the `SecurityMonitor` of Section 5.6, consulted by the coordinator before every assignment. One shipped default deserves a call-out: **resource access is open by default** and restrictable by configuration—a fail-open stance shifting the burden onto the policy writer. The hardened inversion ships as an opt-in strict mode (`SecurityMonitor(..., strict=True)`, propagated by `AgentCoordinator(..., security_strict=True)`; default `False`, fail-open checks unchanged): strict denies unregistered resources, malformed resource-access policies, and unknown data sensitivity, recording each with a `(strict mode)` marker (`tests/core/isa/test_security_failclosed.py`). A paired posture matrix (1,000 tasks per mode, five seeds; Section 5.11) quantifies the difference on the real code paths: fail-open admits all 400 unknown-sensitivity and incomplete-policy tasks that strict denies; both deny the 200 unregistered-resource tasks, strict adding a distinguishable reason class; both also deny the 200 malformed-registry tasks (a malformed registry fails in either posture—the fifth cohort); strict adds zero false denials on the 200 well-specified tasks, and every strict denial is recorded (matrix: supplementary I.2). The 43 functional tests validate the implemented checks under configured policies, not the default posture; nothing in this paper constitutes a security certification. Beyond what is implemented, hardening considerations—not claims about the current system: data privacy (end-to-end data-flow tracking, privacy-preserving computation; Section 6.3.6), secure multi-agent communication channels, audit-log integrity, and externally audited compliance controls all remain future work; the system provides enforcement hooks, not certification.

6.3 Future Research Directions

6.3.1 Theoretical Foundations.

1. **Formal semantics of agent instructions**: consolidating Section 3.1 and Appendix B into one standalone semantics—instruction language, state space, reduction rules, type system; nothing in it is new machinery.
2. **Compositional reasoning**: a parallel-composition rule whose disjointness side condition mirrors the ‖-typing rule of Definition 3.6, enabling modular verification and temporal property preservation (rule in Appendix E.1 of the supplementary material).
3. **Type theory for agent capabilities**: a capability type discipline with Basic/Composite/Emergent kinds and a composition rule $a \circ b : \text{cap}_1 \otimes \text{cap}_2$ (Appendix E.1).

6.3.2 Advanced Instruction Set Evolution.

1. **Self-modifying instructions**—a formal model for instruction evolution, $I_{t+1} = f(I_t, \text{Performance}_t, \text{Context}_t)$ with f left abstract, under semantic-preservation, performance-monotonicity, and safety constraints.
2. **Context-aware optimization**—for each observed context, predict the instruction set's performance against a threshold, and if it falls short, select and accumulate a best rewrite variant. The programmatic sketch (OPTIMIZE-CONTEXT) is listed in Appendix D.9; its primitives are future work, not implemented machinery.
3. **Cross-model instruction mapping**: a universal mapping framework $\phi_{A \to B} : I_A \to I_B$ with semantic preservation, performance bounds, and capability awareness stated as goals, not delivered guarantees.

6.3.3 Theoretical Performance Bounds.

1. **Asymptotic analysis**: deriving cost bounds per composition operator—when does iterative composition i_1^* admit a closed-form cost model, and under what reuse structure does caching yield amortized $O(1)$ effective cost per task? The microbenchmarks (Section 5) measure the constants empirically; the asymptotic theory remains open.
2. **Resource consumption models**: a calibrated model $R(i) = \alpha C_{cpu}(i) + \beta C_{mem}(i) + \gamma C_{io}(i)$ with coefficients fitted against real deployments (Appendix E.1).

6.3.4 Capability Assessment: Extensions. The instruction-level capability assessment subsystem (`CapabilityProfile`, Section 3.5) now ships at the static layer; its wiring gap remains disclosed there. Two extensions remain future work: (i) **benchmark-calibrated**

metrics—correlating scores with *end-to-end* performance [17] [29] and grounding the weights w_k therein; and (ii) **cross-model comparison matrices**, a pairwise framework $\Delta(M_1, M_2)$ over capability dimensions for model selection.

6.3.5 *Emergent Behavior Analysis.*

1. **Collective intelligence modeling**: the open question is a formal metric for when multi-agent composition exceeds the sum of its members' capabilities.

2. **Learning transfer**: knowledge transfer across agents composes Section 3.3's mining-extraction-validation pipeline rather than new machinery—patterns mined from a source agent's traces would be adapted and re-validated before promotion.

6.3.6 *Security and Privacy Extensions.*

1. **Information flow control**: the classical noninterference property $\text{Secure}(P) \iff \forall s_1, s_2 : L(s_1) = L(s_2) \implies L([\![P]\!]s_1) = L([\![P]\!]s_2)$, where L assigns security levels to states and outputs, stated for terminating runs (divergence and channel leakage out of scope; Appendix E.1).

2. **Privacy-preserving computation**: a direction, not a delivered mechanism—if computations over sensitive data were routed through per-step differentially private mechanisms, overall loss would follow a composition theorem over the per-mechanism (ϵ_i, δ_i) parameters. We deliberately state no bound: which theorem applies and which mechanisms the runtime would expose are open design choices; the instruction contract is the natural place to carry such annotations once the mechanisms exist.

None of the frameworks above is implemented today; Sections 6.3.3 and 6.3.4 are stated precisely enough to be falsifiable, while the remaining sketches are programmatic.

7 Conclusion

This paper presented AgentISA, an instruction set architecture that treats LLM agent behavior as a first-class compilable artifact. Our contributions are:

1. A formal instruction set with compositional semantics, a type system, and pre/post-conditions, with sanity results as design rationale (Theorems 3.1–3.3) and semantic-preservation arguments for the optimization and scheduling transformations (Theorems 3.4–3.6).
2. Agentic Calling, which lifts provider-specific function calling into reusable, provider-independent, cacheable agent instructions.
3. Trace-based instruction learning—frequency mining with deterministic action-signature grouping, developer-controlled promotion, and composite replay, with shipped schema-checked parameter binding (unseen-parameter replays exact on 1,050/1,050 cases; constructor-baked parameters excepted; Section 5.11).
4. A compiler and runtime applying classical optimizations—fusion, dead-code elimination, scheduling, and dynamic hot-path specialization—to agent instruction streams.
5. A multi-agent coordination layer with capability-threshold selection and a security monitor, now with an opt-in fail-closed mode.

The evaluation shows the optimizations are real and measurable under controlled conditions: 59.5–79.6% simulated-call reduction from caching, 34.2% instruction-count reduction from static compilation, 65.8% trace shrinkage from hot-path optimization (Sections 5.2–5.5), real-gateway validations of the cache and the compiled pipeline (Sections 5.7–5.7.1), a four-task characterization (Section 5.8), a 100-trace workload census (Section 5.10), six mechanism experiments (Section 5.11), and 43 functional tests for the multi-agent layer. These are microbenchmark and controlled-setting results, not production end-to-end measurements; the threats to validity of Section 5.9 stand.

Implementation status of the contributions. For auditability, the five contributions split by status: *implemented and exercised by the benchmarks* (instruction set and contracts, Agentic Calling, result caching, fusion/DCE/grouping, hot-path detection and adoption, the coordination layer with injected-score capability selection, the security monitor); *implemented with known limits, exercised by the mechanism experiments but not the benchmark arms* (DP tree-pattern selector, EMA hot-path detector, strict cache namespace, strict security mode, parameter-binding composite replay, capability metric extraction); *implemented with known limits, still unexercised by any Section 5 experiment* (the bounded checker; runtime context validation); and *specified, not implemented* (cycle-accurate pipeline scheduling, block-granular trace formation, adaptive recompilation, symbolic-equivalence checking). The per-mechanism status table and the full limitations of each entry are in supplementary F.9; Theorems 3.4 and 3.6 are conditional arguments for the implemented rewrites, Theorem 3.5 covers the shipped unit-slot dispatch, and the specified components carry no verification claim.

AgentISA's longer-term significance is architectural: an instruction-level contract for agent behavior makes agent systems amenable to the discipline that transformed hardware and compiler engineering—precise specification, portability across backends, and semantics-preserving optimization. Establishing these gains on production workloads and mechanically verifying the conditional arguments are the immediate next steps.

Artifact Availability

Full proofs of Theorems 3.1–3.2 and 3.4–3.6, component listings, future-direction sketches, artifact-regeneration commands, the priced-comparison protocol, the four-task characterization detail, and the extension-experiment protocols and tables behind Sections 5.10–5.12 (supplementary Appendices A–I, built from this repository as `docs/AgentISA_supplementary.pdf`) are in the supplementary material; the commands are mirrored in `REPRODUCING.md`. All artifacts are in the repository accompanying this submission (anonymized for review), and every number in Section

5 is generated from the exact repository state whose build produces this PDF (`REPRODUCING.md` records the submission-time `git rev-parse HEAD`, the dependency lock, and the Python/system versions). A fresh clone reproduces the full suite (`python -m pytest tests/ -q`: 1,358 passed, 14 skipped at run time), the Section 5.6 functional-test counts, and, via the Appendix F command lines, every results table from scratch—real-gateway runs included, with a mock-backend fallback for environments without credentials. `REPRODUCING.md` also maps each claim to the implementing module, the exercising test, and the raw-results file, flagging specified-only components; the artifact is inventoried in Appendix F.7, and the figures regenerate from committed results JSONs (`scripts/make_figures.py`).

References

[1] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, et al. 2020. Language Models are Few-Shot Learners. NeurIPS 33. arXiv:2005.14165.

[2] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, et al. 2023. PaLM: Scaling Language Modeling with Pathways. JMLR 24(240):1–113. arXiv:2204.02311.

[3] OpenAI. 2023. GPT-4 Technical Report. arXiv:2303.08774.

[4] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, et al. 2023. LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971.

[5] Jason Wei, Maarten Bosma, Vincent Y. Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, et al. 2022. Finetuned Language Models Are Zero-Shot Learners. ICLR 2022. arXiv:2109.01652.

[6] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, et al. 2022. Training language models to follow instructions with human feedback. NeurIPS 35. arXiv:2203.02155.

[7] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, et al. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 35, pp. 24824–24837. arXiv:2201.11903.

[8] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, Yusuke Iwasawa. 2022. Large Language Models are Zero-Shot Reasoners. NeurIPS 35. arXiv:2205.11916.

[9] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023. arXiv:2210.03629.

[10] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS 36. arXiv:2303.11366.

[11] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, Karthik Narasimhan. 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. NeurIPS 36. arXiv:2305.10601.

[12] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, Anima Anandkumar. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv:2305.16291.

[13] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS 36. arXiv:2302.04761.

[14] Shishir G. Patil, Tianjun Zhang, Xin Wang, Joseph E. Gonzalez. 2023. Gorilla: Large Language Model Connected with Massive APIs. arXiv:2305.15334.

[15] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, et al. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. ICLR 2024. arXiv:2307.16789.

[16] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, Yueting Zhuang. 2023. HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face. NeurIPS 36. arXiv:2303.17580.

[17] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, Joseph E. Gonzalez. 2025. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. ICML 2025. arXiv:2408.12946.

[18] Anthropic. 2024. Introducing the Model Context Protocol. Anthropic announcement, November 25, 2024.

[19] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, et al. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155. (Final version COLM 2024.)

[20] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, et al. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. ICLR 2024. arXiv:2308.00352.

[21] Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, et al. 2024. AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors. ICLR 2024 (Poster). arXiv:2308.10848.

[22] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, et al. 2024. ChatDev: Communicative Agents for Software Development. ACL 2024, pp. 15174–15186. arXiv:2307.07924.

[23] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, Bernard Ghanem. 2023. CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society. NeurIPS 36. arXiv:2303.17760.

[24] Significant Gravitas. 2023. AutoGPT. GitHub repository. https://github.com/Significant-Gravitas/AutoGPT

[25] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, et al. 2023. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. arXiv:2310.03714.

[26] Harrison Chase. 2022. LangChain. GitHub repository. https://github.com/langchain-ai/langchain

[27] Jerry Liu. 2022. LlamaIndex. GitHub repository. https://github.com/run-llama/llama_index

[28] Lingjiao Chen, Matei Zaharia, James Zou. 2023. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv:2305.05176.

[29] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, et al. 2024. AgentBench: Evaluating LLMs as Agents. ICLR 2024. arXiv:2308.03688.

[30] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, Thomas Scialom. 2023. GAIA: a benchmark for General AI Assistants. arXiv:2311.12983. (Final version ICLR 2024.)

[31] Karthik Valmeekam, Matthew Marquez, Alberto Olmo, Sarath Sreedharan, Subbarao Kambhampati. 2023. PlanBench: An Extensible Benchmark for Evaluating Large Language Models on Planning and Reasoning about Change. NeurIPS 2023 Datasets and Benchmarks Track. arXiv:2206.10498.

[32] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world GitHub Issues? ICLR 2024. arXiv:2310.06770.

[33] Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman. 2006. Compilers: Principles, Techniques, and Tools (2nd ed.). Addison-Wesley.

[34] John L. Hennessy, David A. Patterson. 2017. Computer Architecture: A Quantitative Approach (6th ed.). Morgan Kaufmann.

[35] David A. Patterson, Andrew Waterman. 2017. The RISC-V Reader: An Open Architecture Atlas. Strawberry Canyon.

[36] Benjamin C. Pierce. 2002. Types and Programming Languages. MIT Press.

[37] Xavier Leroy. 2009. Formal verification of a realistic compiler. Communications of the ACM 52(7):107–115.

[38] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, et al. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023. arXiv:2309.06180.

[39] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. OSDI 2022, pp. 521–538.

[40] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. NeurIPS 35. arXiv:2205.14135.

[41] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, et al. 2025. Mooncake: Trading More Storage for Less Computation—A KVCache-centric Architecture for Serving LLM Chatbot. FAST 2025, pp. 155–170.

[42] Siddharth Samsi, Dan Zhao, Joseph McDonald, Baolin Li, Adam Michaleas, Michael Jones, et al. 2023. From Words to Watts: Benchmarking the Energy Costs of Large Language Model Inference. arXiv:2310.03003.

[43] Hanne Riis Nielson, Flemming Nielson. 1992. Semantics with Applications: A Formal Introduction. Wiley. (2007 retitled edition: An Appetizer, Springer.)

[44] Michael Sipser. 2012. Introduction to the Theory of Computation, 3rd edition. Cengage Learning. (Church–Turing thesis, Chapter 3.)

[45] Vasanth Bala, Evelyn Duesterwald, Sanjeev Banerjia. 2000. Dynamo: A Transparent Dynamic Optimization System. PLDI 2000. https://doi.org/10.1145/349299.349303

[46] Zilliz and community contributors. 2023. GPTCache: Semantics-Aware Caches for LLM Applications. https://github.com/zilliztech/GPTCache.

[47] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, et al. 2024. SGLang: Efficient Execution of Structured Language Model Programs. NeurIPS 37. arXiv:2312.07104.